Primer Corte – Patrones de Diseño (Semanas 6 a 8)

Integrantes:
Gabriel Augusto Ortega Martínez
Jhonathan David Rojas Molina

Unidades Tecnológicas de Santander
Facultad de Ciencias Naturales e Ingenierías
Ingeniería de sistemas- Snies(101596)

Patrones de software Grupo: A194

Docente: Eliecer Montero Ojeda

Bucaramanga, Octubre 2026

Patrones creacionales aplicados a FábricaViva

Repositorio: https://github.com/GabrielOrtega01/mes-production-system

Contenido

Introducción 3

Objetivos del proyecto 4

Objetivo general 4

Objetivos específicos 4

Patrones creacionales 6

Patrón Singleton (ConfiguracionPlanta) 7

Descripción del patrón 7

Diagrama UML 7

Implementación 8

Prueba de funcionamiento 11

Patrón Factory Method (FabricaOrden) 13

Descripción del patrón 13

Diagrama UML 13

Implementación 14

Prueba de funcionamiento 17

Patrón Builder (GeneradorReportes) 20

Descripción del patrón 20

Diagrama UML 20

Implementación 21

Prueba de funcionamiento 24

Patrón Abstract Factory (FabricaEquipos) 27

Descripción del patrón 27

Diagrama UML 27

Implementación 27

Prueba de funcionamiento 32

Patrón Prototype (EspecificacionPieza) 35

Descripción del patrón 35

Diagrama UML 35

Implementación 36

Prueba de funcionamiento....39
Patrones estructurales....41
Patrón Bridge (AlertaProduccion)....41
Descripción del patrón....41
Diagrama UML....42
Implementación....42
Prueba de funcionamiento....46
Patrón Adapter (AdaptadorErp)....48
Descripción del patrón....48
Diagrama UML....48
Implementación....49
Prueba de funcionamiento....52
Patrón Composite (Jerarquía de Unidades de Producción)....54
Descripción del patrón....54
Diagrama UML....54
Implementación....55
Prueba de funcionamiento....57
Patrón Decorator (Cálculo del Costo de Órdenes de Producción)....62
Descripción del patrón....62
Diagrama UML....62
Implementación....62
Prueba de funcionamiento....64
Referencias....67

Sistema de Control de Producción (MES)

Introducción

En una planta de manufactura casi nunca falta trabajo; lo que suele faltar es información a tiempo. Las órdenes se anotan en papel o en hojas de cálculo que se actualizan cuando alguien se acuerda, una máquina puede pasar media mañana detenida sin que el supervisor lo sepa y, cuando aparece un lote defectuoso, reconstruir qué pasó con cada pieza puede tomar días. Con ese panorama, las decisiones se toman con datos del día anterior, y eso se paga en tiempo perdido, reprocesos y entregas incumplidas.

No es un problema nuevo. MESA International, la asociación que reúne a fabricantes y proveedores de soluciones para manufactura, advertía desde finales de los noventa que la variedad creciente de productos termina desbordando incluso los mejores sistemas manuales y

en papel, y que en planta la información solo es útil si llega en segundos o minutos (MESA International, 1997, p. 4). A las herramientas que atienden esta necesidad se les conoce como sistemas de ejecución de manufactura, o MES por su sigla en inglés: programas que acompañan la producción desde que se lanza una orden hasta que el producto queda terminado (MESA International, 1997, p. 1).

FábricaViva es nuestra propuesta de ese tipo de sistema, y su idea central es sencilla: conectar lo que se planea en la oficina con lo que de verdad ocurre en la planta. Con FábricaViva, el jefe de producción sabe en qué va cada pedido, el área de calidad puede consultar la historia de cualquier pieza, las máquinas reportan por sí solas cuánto han producido o cuándo se detienen, y la gerencia ve al cierre de cada turno qué tan eficiente fue el trabajo, sin esperar a que alguien arme un informe.

Para medir esa eficiencia usamos un indicador muy extendido en la industria, el OEE (Eficiencia General de los Equipos). Nació dentro de la filosofía de Mantenimiento Productivo Total y resume en un solo número tres preguntas que cualquier gerente se hace: ¿la máquina trabajó cuando debía?, ¿lo hizo a la velocidad esperada? y ¿las piezas salieron bien? (Nakajima, 1988). En este proyecto fijamos como meta inicial un OEE de 85 %, valor que aparece en varios de los ejemplos del documento.

Objetivos del proyecto

Objetivo general

Desarrollar un sistema de control de producción que permita mejorar el seguimiento de las actividades realizadas en una planta manufacturera, facilitando el control de las órdenes de producción, la trazabilidad de los productos y el análisis de la información generada durante el proceso productivo.

Objetivos específicos

1. Facilitar el seguimiento de las órdenes de producción y de los productos fabricados, permitiendo conocer el estado y avance de los procesos, así como mantener un registro de la información relacionada con cada producto.
2. Mejorar el control de la información generada durante el proceso de producción, permitiendo registrar aspectos relacionados con el funcionamiento de los equipos, los tiempos de producción y el control de calidad.

3. Proporcionar información organizada que apoye la toma de decisiones, permitiendo identificar posibles dificultades en el proceso productivo y analizar el desempeño general de la producción.

Cumplir estos objetivos no depende solo de lo que el sistema hace, sino también de cómo está construido por dentro. Un programa que crece sin orden se vuelve difícil de cambiar, y en una planta los cambios son constantes: llega un pedido urgente, se compra una máquina de otra marca o la gerencia pide un informe distinto. Por eso, en FábricaViva cuidamos especialmente la forma en que el sistema crea sus componentes internos, y para lograrlo aplicamos los cinco patrones de diseño que se presentan a continuación.

Patrones creacionales

Un patrón de diseño es, según sus autores más reconocidos, una descripción de clases y objetos que colaboran para resolver un problema de diseño general en un determinado contexto (Gamma et al., 2003, p. 4). Dicho de otro modo, es una solución que ya fue probada con éxito y que se puede volver a usar. Ahí está su mayor valor: los patrones sirven para transmitir experiencia entre desarrolladores y facilitan la reutilización del diseño (Medina Veloz et al., 2008, p. 13), siguiendo un esquema de problema, solución y reutilización (Medina Veloz et al., 2008, p. 14).

Los cinco patrones de este documento pertenecen a la familia de los creacionales, que se ocupan de un momento muy concreto: cuándo y cómo el sistema crea sus objetos. Según Gamma et al. (2003), estos patrones proporcionan varias formas de eliminar las referencias explícitas a clases concretas en el código que necesita crear instancias de ellas (p. 77). Llevado a la planta, eso significa algo muy práctico: poder agregar un tipo de orden, un proveedor de maquinaria o un formato de informe sin rehacer lo que ya funciona. La siguiente tabla resume qué situación resuelve cada uno.

Tabla 1

Patrones creacionales aplicados en FábricaViva

Patrón	Situación de la planta que resuelve	Clase principal
Singleton	Existe un único turno activo y una única meta de OEE que todos los módulos deben compartir.	ConfiguracionPlanta
Factory Method	Los pedidos normales y los urgentes se preparan de forma distinta.	FabricaOrden
Builder	El mismo cálculo de eficiencia se entrega resumido o detallado según quién lo lea.	GeneradorReportes
Abstract Factory	Las máquinas son de marcas distintas y sus equipos no se pueden mezclar.	FabricaEquipos
Prototype	Las fichas técnicas de piezas frecuentes se reutilizan como plantillas.	EspecificacionPieza

Nota. Elaboración propia.

Patrón Singleton (ConfiguracionPlanta)

Descripción del patrón

El patrón Singleton garantiza que una clase sólo tenga una instancia, y proporciona un punto de acceso global a ella (Gamma et al., 2003, p. 119). Los autores lo explican con un caso muy parecido al nuestro: un sistema de contabilidad que está dedicado a servir a una única compañía (p. 119). En FábricaViva, la clase ConfiguracionPlanta guarda el nombre de la planta, el turno activo y la meta de OEE, y todos los módulos la consultan por el mismo punto de acceso. Así, cuando supervisión cambia el turno, el cambio lo ven todos al instante.

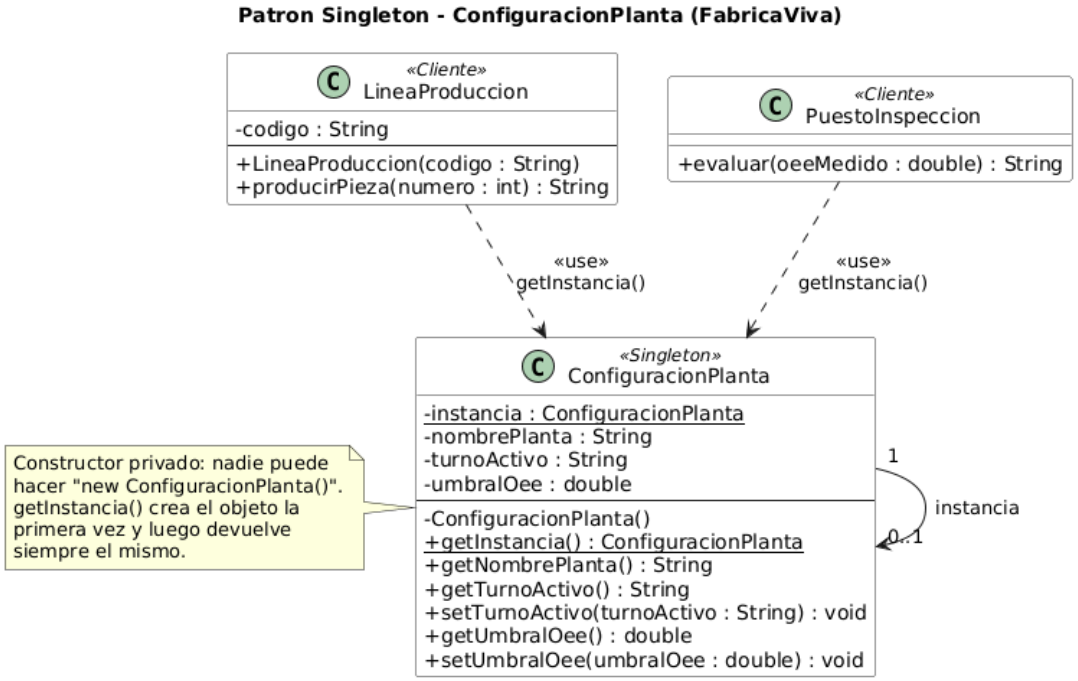
La primera ventaja que señalan Gamma et al. (2003) es el acceso controlado a la única instancia (p. 120): como la propia clase se encarga de crearse, ningún otro módulo puede fabricar una copia por error.

Diagrama UML

En la Figura 1, ConfiguracionPlanta tiene un atributo estático privado, instancia, que aparece subrayado por pertenecer a la clase y no a cada objeto. Ese atributo se asocia con la misma clase con multiplicidad 0..1: antes de la primera llamada no existe ninguna instancia y después existe solo una. El constructor tiene visibilidad privada (–) y getInstancia() es el único método estático público. LineaProduccion y PuestoInspeccion son los clientes: no guardan la configuración como atributo, sino que dependen de ella y la solicitan cada vez que la necesitan.

Figura 1

Diagrama de clases del patrón Singleton en FábricaViva



Nota. Elaborado en PlantUML a partir del archivo `docs/uml/01-singleton.puml`.

Implementación

El patrón se reconoce en tres puntos de ConfiguracionPlanta, marcados en el código con los comentarios (1), (2) y (3) en la Figura 2: el atributo estático que guarda la instancia, el constructor privado que impide usar new desde otras clases y el método getInstancia(), que crea el objeto la primera vez y desde entonces devuelve siempre el mismo. A esta creación diferida Gamma et al. (2003) la llaman inicialización perezosa, porque el objeto no se crea hasta que alguien lo pide por primera vez (p. 121).

Figura 2

Clase ConfiguracionPlanta con los tres elementos del patrón

ConfiguracionPlanta.java

```java
package com.fabricaviva.config;

/**
 * Patrón Singleton.
 * La planta tiene un solo turno activo y una sola meta de OEE en cada momento,
 * así que esta configuración debe existir una única vez y ser compartida por
 * todos los módulos.
 */
public class ConfiguracionPlanta {

    // (1) Atributo estático privado: guarda la única instancia.
    private static ConfiguracionPlanta instancia;

    private final String nombrePlanta;
    private String turnoActivo;
    private double umbralOee;

    // (2) Constructor privado: ninguna otra clase puede usar "new".
    private ConfiguracionPlanta() {
        this.nombrePlanta = "FabricaViva - Planta Bucaramanga";
        this.turnoActivo = "MANANA";
        this.umbralOee = 0.85;
        System.out.println("Creando la unica instancia de ConfiguracionPlanta...");
    }

    // (3) Método estático: único punto de acceso a la instancia.
    public static ConfiguracionPlanta getInstancia() {
        if (instancia == null) {
            instancia = new ConfiguracionPlanta();
        }
        return instancia;
    }

    public String getNombrePlanta() {
        return nombrePlanta;
    }

    public String getTurnoActivo() {
        return turnoActivo;
    }

    public void setTurnoActivo(String turnoActivo) {
        this.turnoActivo = turnoActivo;
    }

    public double getUmbralOee() {
        return umbralOee;
    }

    public void setUmbralOee(double umbralOee) {
        this.umbralOee = umbralOee;
    }
}
```

Nota. Captura del archivo `src/main/java/com/fabricaviva/config/ConfiguracionPlanta.java` del repositorio del proyecto.

Las Figuras 3 y 4 muestran los dos módulos que usan la configuración. Ninguno la crea: ambos llaman a ConfiguracionPlanta.getInstancia() justo antes de leer el turno o la meta.

Figura 3

Módulo LineaProduccion, primer cliente del Singleton

LineaProduccion.java

```java
package com.fabricaviva.produccion;

import com.fabricaviva.config.ConfiguracionPlanta;

/** Primer módulo que usa la configuración: etiqueta cada pieza con el turno. */
public class LineaProduccion {

    private final String codigo;

    public LineaProduccion(String codigo) {
        this.codigo = codigo;
    }

    public String producirPieza(int numero) {
        ConfiguracionPlanta config = ConfiguracionPlanta.getInstancia();
        return "LineaProduccion " + codigo + " -> pieza " + numero
                + " fabricada en turno " + config.getTurnoActivo();
    }
}
```

Nota. Captura del archivo `src/main/java/com/fabricaviva/produccion/LineaProduccion.java` del repositorio del proyecto.

Figura 4

Módulo PuestoInspeccion, segundo cliente del Singleton

PuestoInspeccion.java

```java
package com.fabricaviva.calidad;

import com.fabricaviva.config.ConfiguracionPlanta;

/** Segundo módulo que usa la configuración: compara el OEE con la meta. */
public class PuestoInspeccion {

    public String evaluar(double oeeMedido) {
        ConfiguracionPlanta config = ConfiguracionPlanta.getInstancia();
        String resultado = oeeMedido >= config.getUmbralOee() ? "APROBADO" : "NO APROBADO";
        return "PuestoInspeccion -> OEE " + oeeMedido
                + " (meta " + config.getUmbralOee()
                + ", turno " + config.getTurnoActivo() + "): " + resultado;
    }
}
```

Nota. Captura del archivo `src/main/java/com/fabricaviva/calidad/PuestoInspeccion.java` del repositorio del proyecto.

Una aclaración sobre el alcance: esta versión está pensada para un programa que atiende una tarea a la vez. Si en el futuro varios procesos pidieran la instancia exactamente al mismo tiempo, habría que proteger getInstancia(), por ejemplo declarándolo synchronized.

Prueba de funcionamiento

La clase PruebaSingleton (Figura 5) hace tres comprobaciones, cuyo resultado aparece en la Figura 6.

Figura 5

Clase PruebaSingleton

PruebaSingleton.java

```java
package com.fabricaviva;

import com.fabricaviva.calidad.PuestoInspeccion;
import com.fabricaviva.config.ConfiguracionPlanta;
import com.fabricaviva.produccion.LineaProduccion;

public class PruebaSingleton {

    public static void main(String[] args) {
        System.out.println("== Prueba 1: se pide la configuracion tres veces ==");
        ConfiguracionPlanta a = ConfiguracionPlanta.getInstancia();
        ConfiguracionPlanta b = ConfiguracionPlanta.getInstancia();
        ConfiguracionPlanta c = ConfiguracionPlanta.getInstancia();

        System.out.println("\n== Prueba 2: las tres referencias son el mismo objeto ==");
        System.out.println("hashCode a: " + a.hashCode());
        System.out.println("hashCode b: " + b.hashCode());
        System.out.println("hashCode c: " + c.hashCode());
        System.out.println("a == b ? " + (a == b) + " | b == c ? " + (b == c));

        System.out.println("\n== Prueba 3: dos modulos comparten la misma configuracion ==");
        LineaProduccion linea = new LineaProduccion("CNC-01");
        PuestoInspeccion inspeccion = new PuestoInspeccion();
        System.out.println(linea.producirPieza(1));
        System.out.println(inspeccion.evaluar(0.91));

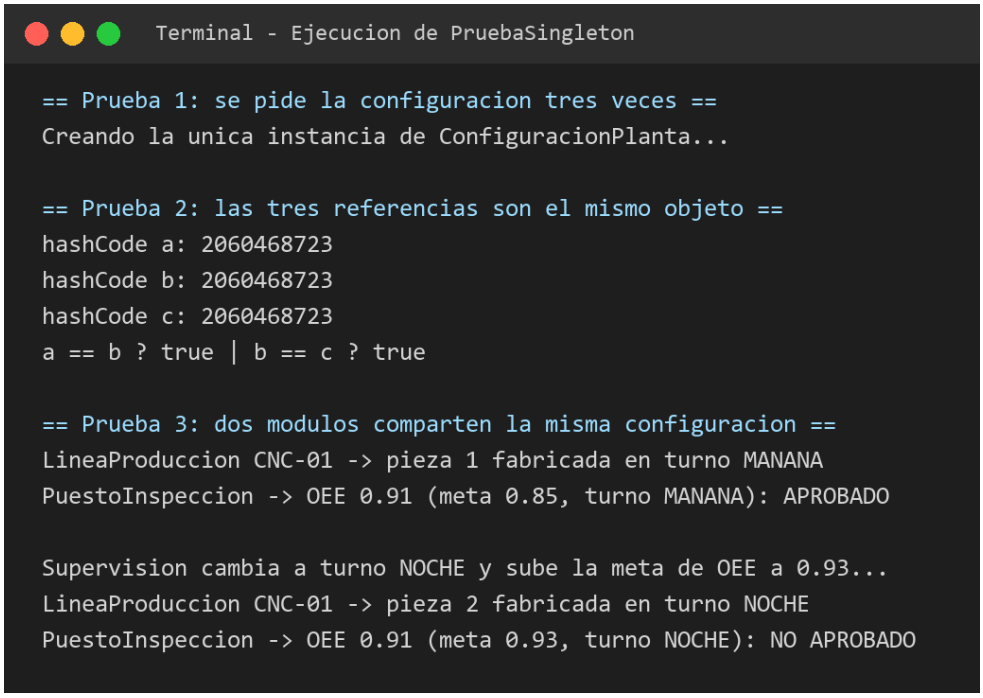
        System.out.println("\nSupervision cambia a turno NOCHE y sube la meta de OEE a 0.93...");
        a.setTurnoActivo("NOCHE");
        a.setUmbralOee(0.93);

        System.out.println(linea.producirPieza(2));
        System.out.println(inspeccion.evaluar(0.91));
    }
}
```

Nota. Captura del archivo `src/main/java/com/fabricaviva/PruebaSingleton.java` del repositorio del proyecto.

Figura 6

Resultado de la ejecución de PruebaSingleton



Nota. Salida obtenida al ejecutar `PruebaSingleton`.

En la primera se pide la configuración tres veces seguidas y el mensaje Creando la unica instancia de ConfiguracionPlanta... aparece una sola vez, lo que confirma que el objeto se creó únicamente en la primera llamada.

En la segunda se comparan las tres referencias: los hashCode son idénticos y tanto a == b como b == c devuelven true, es decir, las tres variables apuntan al mismo objeto en memoria.

En la tercera se pone a trabajar a los dos módulos. Con la meta en 0.85, una pieza con OEE de 0.91 queda APROBADA en el turno MANANA. Luego supervisión cambia a turno NOCHE y sube la meta a 0.93 una sola vez, a través de la variable a. Sin tocar la línea ni el puesto de inspección, los dos reflejan el cambio: la pieza 2 queda registrada en turno NOCHE y el mismo 0.91 pasa a NO APROBADO.

Patrón Factory Method (FabricaOrden)

Descripción del patrón

Según Gamma et al. (2003), el Factory Method define una interfaz para crear un objeto, pero deja que sean las subclases quienes decidan qué clase instanciar (p. 99). Su principal beneficio es que estos métodos eliminan la necesidad de ligar clases específicas de la aplicación a nuestro código (p. 101).

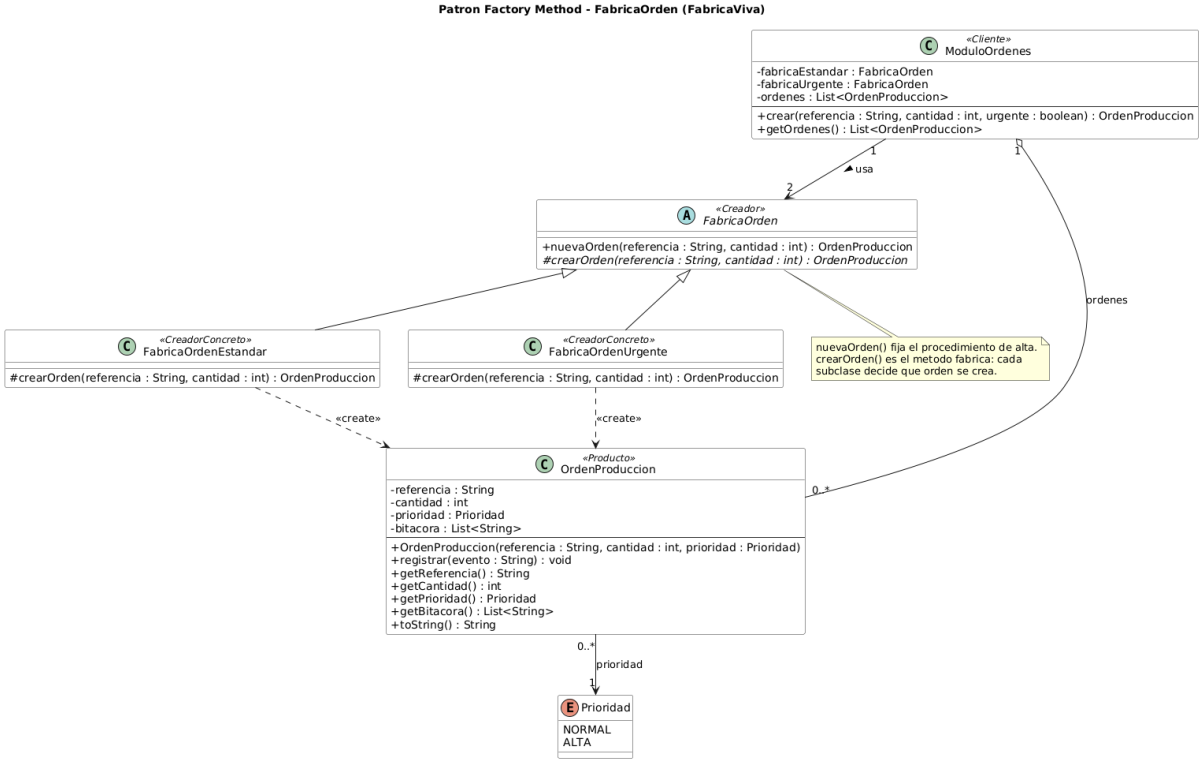
En FábricaViva, la clase abstracta FabricaOrden cumple el papel de creador. Su método nuevaOrden() define el procedimiento que toda orden sigue al darse de alta, pero delega en el método fábrica crearOrden() la decisión de qué orden construir. FabricaOrdenEstandar crea órdenes con prioridad NORMAL y FabricaOrdenUrgente con prioridad ALTA. ModuloOrdenes, que es quien recibe los pedidos, solo conoce a FabricaOrden: le basta con indicar si el pedido es urgente.

Diagrama UML

En la Figura 7, FabricaOrden aparece en cursiva por ser abstracta, igual que su método crearOrden(), que tiene visibilidad protegida (#) porque solo lo usan la clase y sus subclases. Las dos fábricas concretas heredan de ella y crean (create) objetos OrdenProduccion. ModuloOrdenes se asocia con dos fábricas (multiplicidad 2) y agrupa de cero a muchas órdenes. Cada orden tiene exactamente una Prioridad.

Figura 7

Diagrama de clases del patrón Factory Method en FábricaViva



Nota. Elaborado en PlantUML a partir del archivo `docs/uml/02-factory-method.puml`.

Implementación

La Figura 8 muestra FabricaOrden: nuevaOrden() llama a crearOrden() y después anota en la bitácora qué fábrica generó la orden. Las Figuras 9 y 10 muestran las dos fábricas concretas; la urgente, además de fijar la prioridad ALTA, deja la nota de que la orden pasa delante del plan vigente.

Figura 8

Clase abstracta FabricaOrden con el método fábrica crearOrden()

FabricaOrden.java

```java
package com.fabricaviva.ordenes;

/**
 * Patrón Factory Method (Creador).
 * Define el procedimiento para dar de alta una orden, pero deja que cada
 * subclase decida qué tipo de orden se crea.
 */
public abstract class FabricaOrden {

    public OrdenProduccion nuevaOrden(String referencia, int cantidad) {
        OrdenProduccion orden = crearOrden(referencia, cantidad);
        orden.registrar("Creada por " + getClass().getSimpleName());
        return orden;
    }

    // Método fábrica: cada subclase lo implementa a su manera.
    protected abstract OrdenProduccion crearOrden(String referencia, int cantidad);
}
```

Nota. Captura del archivo `src/main/java/com/fabricaviva/ordenes/FabricaOrden.java` del repositorio del proyecto.

Figura 9

Fábrica concreta FabricaOrdenEstandar

FabricaOrdenEstandar.java

```java
package com.fabricaviva.ordenes;

/** Órdenes del plan normal de producción. */
public class FabricaOrdenEstandar extends FabricaOrden {

    @Override
    protected OrdenProduccion crearOrden(String referencia, int cantidad) {
        return new OrdenProduccion(referencia, cantidad, Prioridad.NORMAL);
    }
}
```

Nota. Captura del archivo `src/main/java/com/fabricaviva/ordenes/FabricaOrdenEstandar.java` del repositorio del proyecto.

Figura 10

Fábrica concreta FabricaOrdenUrgente

FabricaOrdenUrgente.java

```java
package com.fabricaviva.ordenes;

/** Pedidos urgentes que pasan por delante del plan vigente. */
public class FabricaOrdenUrgente extends FabricaOrden {

    @Override
    protected OrdenProduccion crearOrden(String referencia, int cantidad) {
        OrdenProduccion orden = new OrdenProduccion(referencia, cantidad, Prioridad.ALTA);
        orden.registrar("Marcada urgente: pasa delante del plan vigente");
        return orden;
    }
}
```

Nota. Captura del archivo `src/main/java/com/fabricaviva/ordenes/FabricaOrdenUrgente.java` del repositorio del proyecto.

En la Figura 11, ModuloOrdenes elige la fábrica en una sola línea y nunca menciona las clases concretas. Si mañana la planta necesita órdenes de muestra, bastará con crear una nueva subclase de FabricaOrden, sin modificar las que ya existen.

Figura 11

Clase ModuloOrdenes, cliente del Factory Method

ModuloOrdenes.java

```java
package com.fabricaviva.ordenes;

import java.util.ArrayList;
import java.util.List;

/**
 * Cliente del Factory Method. Solo conoce la abstracción FabricaOrden;
 * no sabe cómo se arma por dentro cada tipo de orden.
 */
public class ModuloOrdenes {

    private final FabricaOrden fabricaEstandar = new FabricaOrdenEstandar();
    private final FabricaOrden fabricaUrgente = new FabricaOrdenUrgente();
    private final List<OrdenProduccion> ordenes = new ArrayList<>();

    public OrdenProduccion crear(String referencia, int cantidad, boolean urgente) {
        FabricaOrden fabrica = urgente ? fabricaUrgente : fabricaEstandar;
        OrdenProduccion orden = fabrica.nuevaOrden(referencia, cantidad);
        ordenes.add(orden);
        return orden;
    }

    public List<OrdenProduccion> getOrdenes() {
        return ordenes;
    }
}
```

Nota. Captura del archivo `src/main/java/com/fabricaviva/ordenes/ModuloOrdenes.java` del repositorio del proyecto.

Prueba de funcionamiento

PruebaFactoryMethod (Figura 12) tiene dos partes. En la primera se usa cada fábrica directamente: la estándar produce la orden REF-1001 con prioridad NORMAL y la urgente produce la REF-2002 con prioridad ALTA, cada una con su propia bitácora.

Figura 12

Clase PruebaFactoryMethod

PruebaFactoryMethod.java

```java
package com.fabricaviva;

import com.fabricaviva.ordenes.FabricaOrden;
import com.fabricaviva.ordenes.FabricaOrdenEstandar;
import com.fabricaviva.ordenes.FabricaOrdenUrgente;
import com.fabricaviva.ordenes.ModuloOrdenes;
import com.fabricaviva.ordenes.OrdenProduccion;

public class PruebaFactoryMethod {

    public static void main(String[] args) {
        System.out.println("== Prueba 1: cada fabrica decide que orden crear ==");
        FabricaOrden estandar = new FabricaOrdenEstandar();
        FabricaOrden urgente = new FabricaOrdenUrgente();

        OrdenProduccion o1 = estandar.nuevaOrden("REF-1001", 500);
        OrdenProduccion o2 = urgente.nuevaOrden("REF-2002", 120);
        System.out.println(o1 + " | bitacora: " + o1.getBitacora());
        System.out.println(o2 + " | bitacora: " + o2.getBitacora());

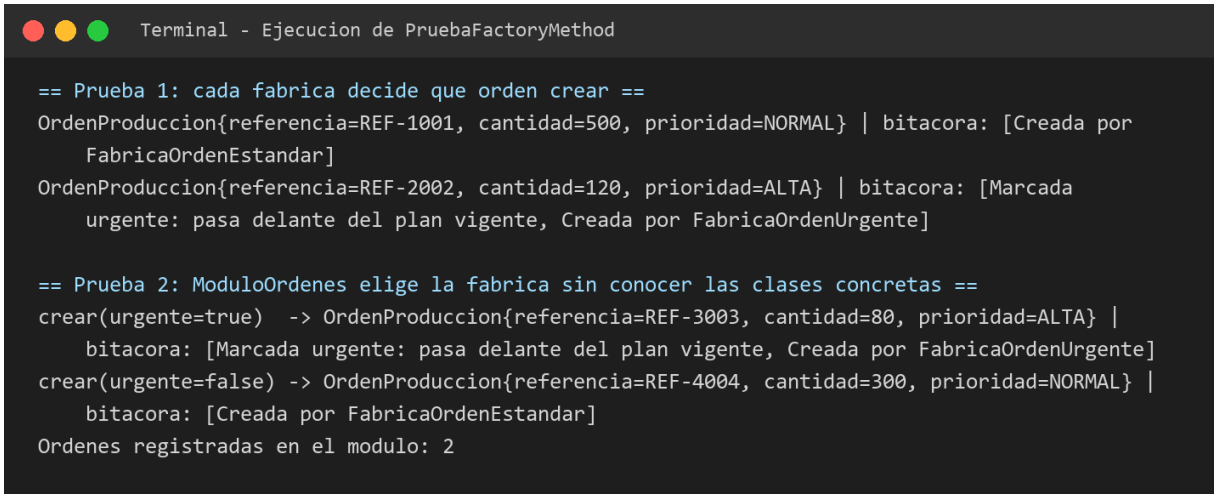
        System.out.println("\n== Prueba 2: ModuloOrdenes elige la fabrica sin conocer las clases concretas ==");
        ModuloOrdenes modulo = new ModuloOrdenes();
        OrdenProduccion o3 = modulo.crear("REF-3003", 80, true);
        OrdenProduccion o4 = modulo.crear("REF-4004", 300, false);
        System.out.println("crear(urgente=true)  -> " + o3 + " | bitacora: " + o3.getBitacora());
        System.out.println("crear(urgente=false) -> " + o4 + " | bitacora: " + o4.getBitacora());
        System.out.println("Ordenes registradas en el modulo: " + modulo.getOrdenes().size());
    }
}
```

Nota. Captura del archivo `src/main/java/com/fabricaviva/PruebaFactoryMethod.java` del repositorio del proyecto.

En la segunda parte se llama al método real ModuloOrdenes.crear(), primero con urgente=true y luego con urgente=false. Como se ve en la Figura 13, el módulo entregó una orden ALTA y otra NORMAL, cada una con la bitácora de la fábrica que la creó, y dejó registradas las dos órdenes, sin conocer en ningún momento cómo se construye cada tipo.

Figura 13

Resultado de la ejecución de PruebaFactoryMethod



Nota. Salida obtenida al ejecutar `PruebaFactoryMethod`.

Patrón Builder (GeneradorReportes)

Descripción del patrón

El Builder separa la construcción de un objeto complejo de su representación, de forma que el mismo proceso de construcción pueda crear diferentes representaciones (Gamma et al., 2003, p. 89). En él participan un director, que conduce los pasos; un constructor, que define cómo se arma cada parte, y el producto final (Gamma et al., 2003, pp. 90-91).

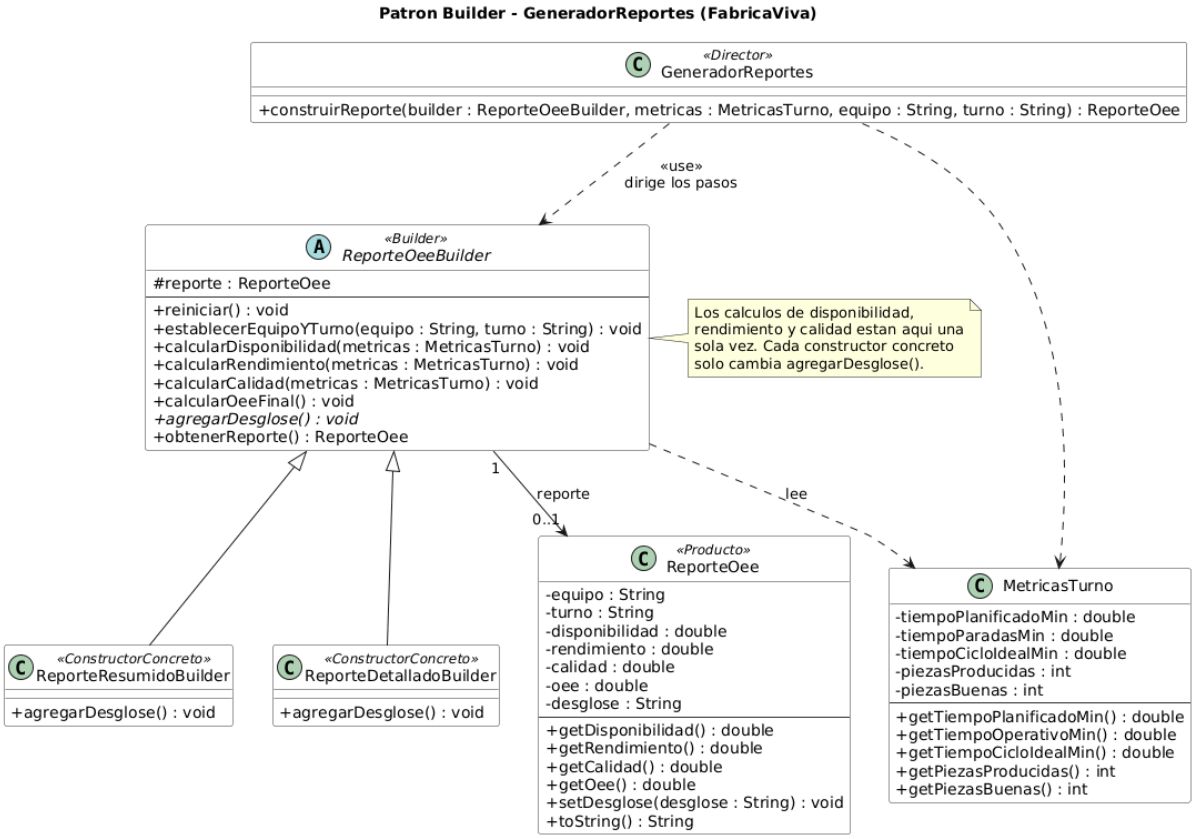
En FábricaViva, GeneradorReportes es el director y siempre sigue la misma secuencia de pasos. ReporteOeeBuilder es el constructor abstracto y concentra los cálculos de disponibilidad, rendimiento, calidad y OEE, de modo que se escriben una sola vez. Los constructores concretos solo deciden cómo termina el reporte: ReporteResumidoBuilder lo deja con el OEE final y ReporteDetalladoBuilder le agrega el desglose. Gamma et al. (2003) destacan que así se puede variar la representación interna de un producto, porque para obtener un reporte nuevo basta con definir otro constructor (p. 91).

Diagrama UML

En la Figura 14, GeneradorReportes depende de ReporteOeeBuilder, que recibe como parámetro en construirReporte(). ReporteOeeBuilder es abstracta; su atributo protegido reporte (#) se asocia con cero o un ReporteOee, porque antes de llamar a reiniciar() el reporte todavía no existe. agregarDesglose() aparece en cursiva por ser el único paso abstracto, y los dos constructores concretos heredan de la clase para implementar solo ese paso. MetricasTurno contiene los datos crudos del turno que se usan en los cálculos.

Figura 14

Diagrama de clases del patrón Builder en FábricaViva



Nota. Elaborado en PlantUML a partir del archivo `docs/uml/03-builder.puml`.

Implementación

En la Figura 15 están los cálculos compartidos, cada uno con un comentario que explica la fórmula en lenguaje sencillo. Las Figuras 16 y 17 muestran que los constructores concretos son pequeños: el resumido deja vacío agregarDesglose() y el detallado arma el texto con los tres porcentajes.

Figura 15

Constructor abstracto ReporteOeeBuilder con los cálculos del OEE

ReporteOeeBuilder.java

```java
package com.fabricaviva.reportes;

/**
 * Patrón Builder (Constructor).
 * Los pasos de cálculo son iguales para todos los reportes y viven aquí una
 * sola vez. Cada constructor concreto solo decide cómo presentar el resultado.
 */
public abstract class ReporteOeeBuilder {

    protected ReporteOee reporte;

    public void reiniciar() {
        reporte = new ReporteOee();
    }

    public void establecerEquipoYTurno(String equipo, String turno) {
        reporte.setEquipo(equipo);
        reporte.setTurno(turno);
    }

    // Disponibilidad = tiempo que la máquina trabajó / tiempo planificado
    public void calcularDisponibilidad(MetricasTurno metricas) {
        reporte.setDisponibilidad(metricas.getTiempoOperativoMin() / metricas.getTiempoPlanificadoMin());
    }

    // Rendimiento = tiempo ideal para las piezas hechas / tiempo que la máquina trabajó
    public void calcularRendimiento(MetricasTurno metricas) {
        double tiempoIdeal = metricas.getTiempoCicloIdealMin() * metricas.getPiezasProducidas();
        reporte.setRendimiento(tiempoIdeal / metricas.getTiempoOperativoMin());
    }

    // Calidad = piezas buenas / piezas producidas
    public void calcularCalidad(MetricasTurno metricas) {
        reporte.setCalidad((double) metricas.getPiezasBuenas() / metricas.getPiezasProducidas());
    }

    public void calcularOeeFinal() {
        reporte.setOee(reporte.getDisponibilidad() * reporte.getRendimiento() * reporte.getCalidad());
    }

    // Único paso que cambia según el tipo de reporte.
    public abstract void agregarDesglose();

    public ReporteOee obtenerReporte() {
        return reporte;
    }
}
```

Nota. Captura del archivo `src/main/java/com/fabricaviva/reportes/ReporteOeeBuilder.java` del repositorio del proyecto.

Figura 16

Constructor concreto ReporteResumidoBuilder

ReporteResumidoBuilder.java

```java
package com.fabricaviva.reportes;

/** Para el supervisor: solo le interesa el OEE final. */
public class ReporteResumidoBuilder extends ReporteOeeBuilder {

    @Override
    public void agregarDesglose() {
        // El resumen no lleva desglose.
    }
}
```

Nota. Captura del archivo `src/main/java/com/fabricaviva/reportes/ReporteResumidoBuilder.java` del repositorio del proyecto.

Figura 17

Constructor concreto ReporteDetalladoBuilder

ReporteDetalladoBuilder.java

```java
package com.fabricaviva.reportes;

import java.util.Locale;

/** Para el analista de mejora continua: necesita ver dónde se pierde eficiencia. */
public class ReporteDetalladoBuilder extends ReporteOeeBuilder {

    @Override
    public void agregarDesglose() {
        reporte.setDesglose(String.format(Locale.US,
                "Disponibilidad: %.2f%% | Rendimiento: %.2f%% | Calidad: %.2f%%",
                reporte.getDisponibilidad() * 100,
                reporte.getRendimiento() * 100,
                reporte.getCalidad() * 100));
    }
}
```

Nota. Captura del archivo `src/main/java/com/fabricaviva/reportes/ReporteDetalladoBuilder.java` del repositorio del proyecto.

La Figura 18 muestra al director. construirReporte() ejecuta siempre los mismos pasos, en el mismo orden, sin saber qué tipo de constructor recibió.

Figura 18

Director GeneradorReportes

GeneradorReportes.java

```java
package com.fabricaviva.reportes;

/**
 * Director del patrón Builder: siempre sigue la misma secuencia de pasos,
 * sin saber qué tipo de reporte resultará.
 */
public class GeneradorReportes {

    public ReporteOee construirReporte(ReporteOeeBuilder builder, MetricasTurno metricas,
                                       String equipo, String turno) {
        builder.reiniciar();
        builder.establecerEquipoYTurno(equipo, turno);
        builder.calcularDisponibilidad(metricas);
        builder.calcularRendimiento(metricas);
        builder.calcularCalidad(metricas);
        builder.calcularOeeFinal();
        builder.agregarDesglose();
        return builder.obtenerReporte();
    }
}
```

Nota. Captura del archivo `src/main/java/com/fabricaviva/reportes/GeneradorReportes.java` del repositorio del proyecto.

Prueba de funcionamiento

PruebaBuilder (Figura 19) registra un turno de 480 minutos con 24 minutos de paradas, un tiempo de ciclo ideal de medio minuto por pieza y 820 piezas producidas, de las cuales 804 salieron buenas. Con esos datos arma los dos reportes usando el mismo GeneradorReportes.

Figura 19

Clase PruebaBuilder

PruebaBuilder.java

```java
package com.fabricaviva;

import com.fabricaviva.reportes.GeneradorReportes;
import com.fabricaviva.reportes.MetricasTurno;
import com.fabricaviva.reportes.ReporteDetalladoBuilder;
import com.fabricaviva.reportes.ReporteOee;
import com.fabricaviva.reportes.ReporteResumidoBuilder;

public class PruebaBuilder {

    public static void main(String[] args) {
        // Turno de 480 min con 24 min de paradas, ciclo ideal de 0.5 min,
        // 820 piezas producidas de las cuales 804 salieron buenas.
        MetricasTurno metricas = new MetricasTurno(480, 24, 0.5, 820, 804);
        GeneradorReportes generador = new GeneradorReportes();

        ReporteOee resumen = generador.construirReporte(
                new ReporteResumidoBuilder(), metricas, "CNC-01", "MANANA");
        ReporteOee detallado = generador.construirReporte(
                new ReporteDetalladoBuilder(), metricas, "CNC-01", "MANANA");

        System.out.println("== Reporte resumido (supervisor) ==");
        System.out.println(resumen);

        System.out.println("\n== Reporte detallado (analista) ==");
        System.out.println(detallado);

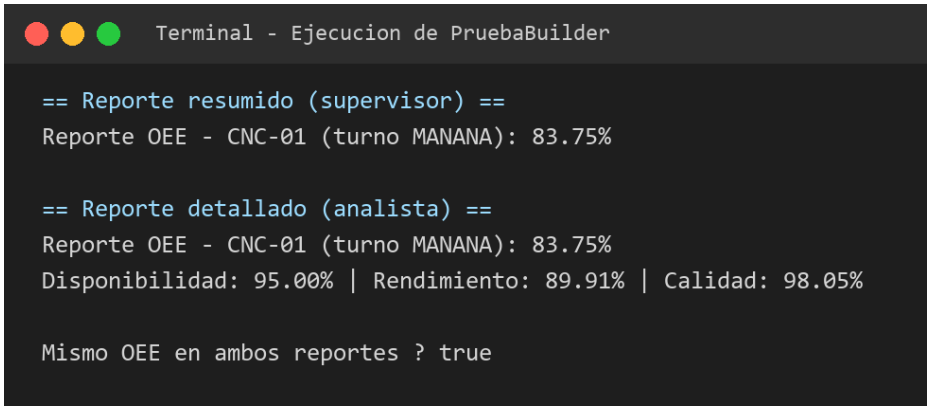
        System.out.println("\nMismo OEE en ambos reportes ? " + (resumen.getOee() == detallado.getOee()));
    }
}
```

Nota. Captura del archivo `src/main/java/com/fabricaviva/PruebaBuilder.java` del repositorio del proyecto.

En la Figura 20 se ve que ambos reportes llegan al mismo OEE, 83.75 %, y la última línea lo confirma con true. La única diferencia es que el reporte detallado muestra de dónde sale ese número: 95 % de disponibilidad, 89.91 % de rendimiento y 98.05 % de calidad. Para el analista la lectura es inmediata: el mayor margen de mejora está en la velocidad de la máquina.

Figura 20

Resultado de la ejecución de PruebaBuilder



Nota. Salida obtenida al ejecutar `PruebaBuilder`.

Patrón Abstract Factory (FabricaEquipos)

Descripción del patrón

El Abstract Factory proporciona una interfaz para crear familias de objetos relacionados o que dependen entre sí, sin especificar sus clases concretas (Gamma et al., 2003, p. 79). Entre sus ventajas, los autores señalan que facilita el intercambio de familias de productos y que promueve la consistencia entre ellos (p. 81), que es justo lo que necesita la planta.

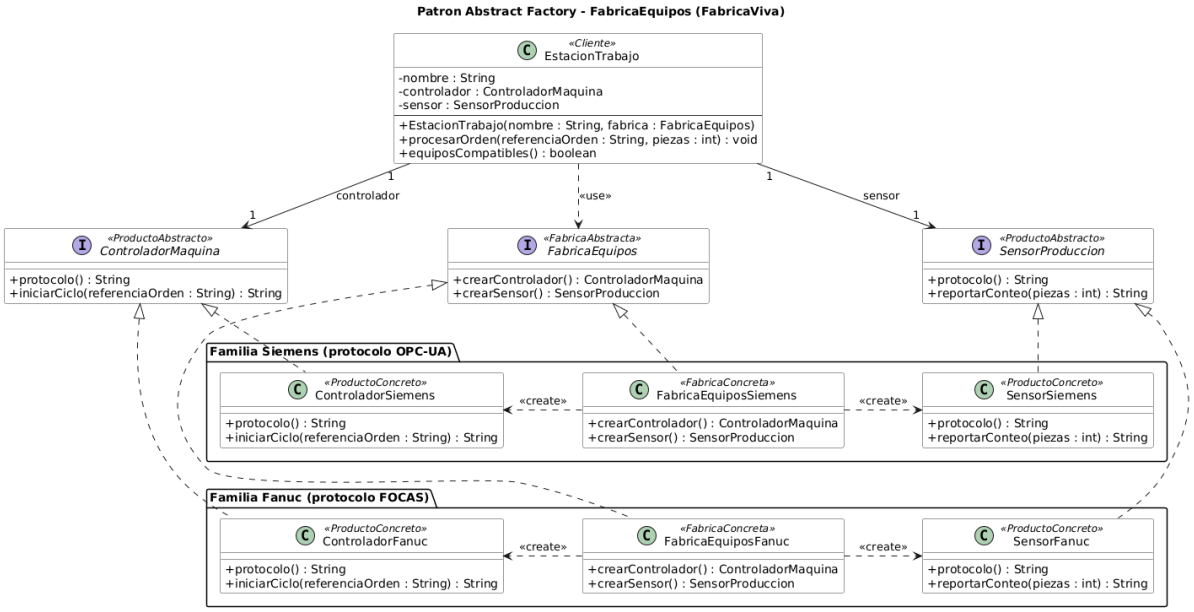
En FábricaViva, la interfaz FabricaEquipos declara dos operaciones: crear un controlador y crear un sensor. FabricaEquiposSiemens y FabricaEquiposFanuc son las fábricas concretas, y cada una entrega únicamente equipos de su marca. EstacionTrabajo, el cliente, recibe una fábrica al crearse y arma con ella sus dos equipos, sin mencionar nunca a Siemens ni a Fanuc.

Diagrama UML

La Figura 21 se lee en tres niveles. Arriba está el cliente, EstacionTrabajo, que guarda exactamente un controlador y un sensor (multiplicidad 1) y usa la fábrica abstracta. En el medio están las tres interfaces: FabricaEquipos y los dos productos abstractos. Abajo hay una fila por familia; en cada una, la fábrica concreta crea (create) el controlador y el sensor de su misma marca, y cada clase realiza (línea punteada con triángulo) la interfaz que le corresponde.

Figura 21

Diagrama de clases del patrón Abstract Factory en FábricaViva



Nota. Elaborado en PlantUML a partir del archivo `docs/uml/04-abstract-factory.puml`.

Implementación

Las Figuras 22, 23 y 24 muestran las tres interfaces que definen la familia de productos.

Figura 22

Interfaz FabricaEquipos (fábrica abstracta)

FabricaEquipos.java

```java
package com.fabricaviva.equipos;

/**
 * Patrón Abstract Factory.
 * Crea la familia completa de equipos de un mismo fabricante, de modo que el
 * controlador y el sensor de una estación siempre hablen el mismo protocolo.
 */
public interface FabricaEquipos {

    ControladorMaquina crearControlador();

    SensorProduccion crearSensor();
}
```

Nota. Captura del archivo `src/main/java/com/fabricaviva/equipos/FabricaEquipos.java` del repositorio del proyecto.

Figura 23

Interfaz ControladorMaquina (producto abstracto)

ControladorMaquina.java

```java
package com.fabricaviva.equipos;

/** Producto abstracto A: el controlador que arranca la máquina. */
public interface ControladorMaquina {

    String protocolo();

    String iniciarCiclo(String referenciaOrden);
}
```

Nota. Captura del archivo `src/main/java/com/fabricaviva/equipos/ControladorMaquina.java` del repositorio del proyecto.

Figura 24

Interfaz SensorProduccion (producto abstracto)

SensorProduccion.java

```java
package com.fabricaviva.equipos;

/** Producto abstracto B: el sensor que cuenta las piezas fabricadas. */
public interface SensorProduccion {

    String protocolo();

    String reportarConteo(int piezas);
}
```

Nota. Captura del archivo `src/main/java/com/fabricaviva/equipos/SensorProduccion.java` del repositorio del proyecto.

En la Figura 25, FabricaEquiposSiemens devuelve siempre un ControladorSiemens y un SensorSiemens. La fábrica de Fanuc tiene la misma estructura con sus propias clases y está disponible en el repositorio. La Figura 26 muestra uno de los productos concretos, que responde con el protocolo de su marca.

Figura 25

Fábrica concreta FabricaEquiposSiemens

FabricaEquiposSiemens.java

```java
package com.fabricaviva.equipos.siemens;

import com.fabricaviva.equipos.ControladorMaquina;
import com.fabricaviva.equipos.FabricaEquipos;
import com.fabricaviva.equipos.SensorProduccion;

/** Fábrica concreta: toda la familia de equipos Siemens. */
public class FabricaEquiposSiemens implements FabricaEquipos {

    @Override
    public ControladorMaquina crearControlador() {
        return new ControladorSiemens();
    }

    @Override
    public SensorProduccion crearSensor() {
        return new SensorSiemens();
    }
}
```

Nota. Captura del archivo `src/main/java/com/fabricaviva/equipos/siemens/FabricaEquiposSiemens.java` del repositorio del proyecto.

Figura 26

Producto concreto ControladorSiemens

ControladorSiemens.java

```java
package com.fabricaviva.equipos.siemens;

import com.fabricaviva.equipos.ControladorMaquina;

public class ControladorSiemens implements ControladorMaquina {

    @Override
    public String protocolo() {
        return "OPC-UA";
    }

    @Override
    public String iniciarCiclo(String referenciaOrden) {
        return "[OPC-UA] Siemens inicia ciclo de la orden " + referenciaOrden;
    }
}
```

Nota. Captura del archivo `src/main/java/com/fabricaviva/equipos/siemens/ControladorSiemens.java` del repositorio del proyecto.

Por último, en la Figura 27, EstacionTrabajo obtiene sus equipos de la fábrica que recibe en el constructor y ofrece el método equiposCompatibles() para comprobar que ambos hablan el mismo protocolo.

Figura 27

Clase EstacionTrabajo, cliente del Abstract Factory

EstacionTrabajo.java

```java
package com.fabricaviva.equipos;

/**
 * Cliente del Abstract Factory. Recibe una fábrica y arma sus equipos con ella;
 * nunca menciona Siemens ni Fanuc.
 */
public class EstacionTrabajo {

    private final String nombre;
    private final ControladorMaquina controlador;
    private final SensorProduccion sensor;

    public EstacionTrabajo(String nombre, FabricaEquipos fabrica) {
        this.nombre = nombre;
        this.controlador = fabrica.crearControlador();
        this.sensor = fabrica.crearSensor();
    }

    public void procesarOrden(String referenciaOrden, int piezas) {
        System.out.println(nombre + ":");
        System.out.println("  " + controlador.iniciarCiclo(referenciaOrden));
        System.out.println("  " + sensor.reportarConteo(piezas));
    }

    public boolean equiposCompatibles() {
        return controlador.protocolo().equals(sensor.protocolo());
    }
}
```

Nota. Captura del archivo `src/main/java/com/fabricaviva/equipos/EstacionTrabajo.java` del repositorio del proyecto.

Prueba de funcionamiento

La prueba (Figura 28) crea dos estaciones: el torno con la fábrica de Siemens y la fresadora con la de Fanuc.

Figura 28

Clase PruebaAbstractFactory

PruebaAbstractFactory.java

```java
package com.fabricaviva;

import com.fabricaviva.equipos.EstacionTrabajo;
import com.fabricaviva.equipos.fanuc.FabricaEquiposFanuc;
import com.fabricaviva.equipos.siemens.FabricaEquiposSiemens;

public class PruebaAbstractFactory {

    public static void main(String[] args) {
        System.out.println("== Prueba 1: cada estacion recibe la familia de equipos de una marca ==");
        EstacionTrabajo torno = new EstacionTrabajo("Estacion Torno", new FabricaEquiposSiemens());
        EstacionTrabajo fresa = new EstacionTrabajo("Estacion Fresadora", new FabricaEquiposFanuc());

        torno.procesarOrden("REF-1001", 250);
        fresa.procesarOrden("REF-2002", 120);

        System.out.println("\n== Prueba 2: controlador y sensor siempre hablan el mismo protocolo ==");
        System.out.println("Estacion Torno compatible ? " + torno.equiposCompatibles());
        System.out.println("Estacion Fresadora compatible ? " + fresa.equiposCompatibles());

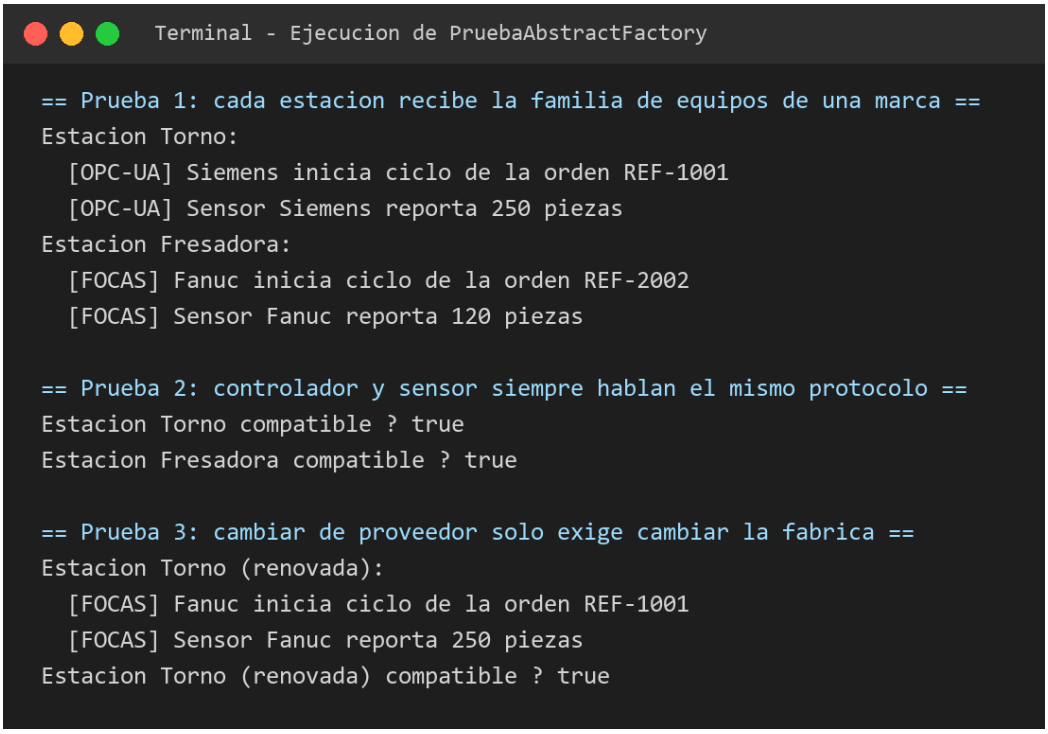
        System.out.println("\n== Prueba 3: cambiar de proveedor solo exige cambiar la fabrica ==");
        EstacionTrabajo tornoRenovado = new EstacionTrabajo("Estacion Torno (renovada)", new FabricaEquiposFanuc());
        tornoRenovado.procesarOrden("REF-1001", 250);
        System.out.println("Estacion Torno (renovada) compatible ? " + tornoRenovado.equiposCompatibles());
    }
}
```

Nota. Captura del archivo `src/main/java/com/fabricaviva/PruebaAbstractFactory.java` del repositorio del proyecto.

En la Figura 29 se aprecia que cada estación trabaja con equipos de una sola marca: los mensajes del torno salen con [OPC-UA] y los de la fresadora con [FOCAS], y las dos responden true a la verificación de compatibilidad. La tercera parte simula que la empresa renueva los equipos del torno con otro proveedor: basta con entregarle la fábrica de Fanuc. La estación funciona igual, ahora con sus dos equipos en FOCAS, sin que se haya modificado una sola línea de EstacionTrabajo.

Figura 29

Resultado de la ejecución de PruebaAbstractFactory



Nota. Salida obtenida al ejecutar `PruebaAbstractFactory`.

Patrón Prototype (EspecificacionPieza)

Descripción del patrón

El Prototype especifica los tipos de objetos a crear por medio de una instancia prototípica, y crea nuevos objetos copiando dicho prototipo (Gamma et al., 2003, p. 109). Sus participantes son un prototipo, que declara una interfaz para clonarse, los prototipos concretos que la implementan y un cliente que crea objetos pidiéndole al prototipo que se clone (p. 111). Entre sus beneficios, los autores mencionan la posibilidad de especificar nuevos objetos modificando valores (p. 111), que es exactamente el caso del eje con tolerancia especial.

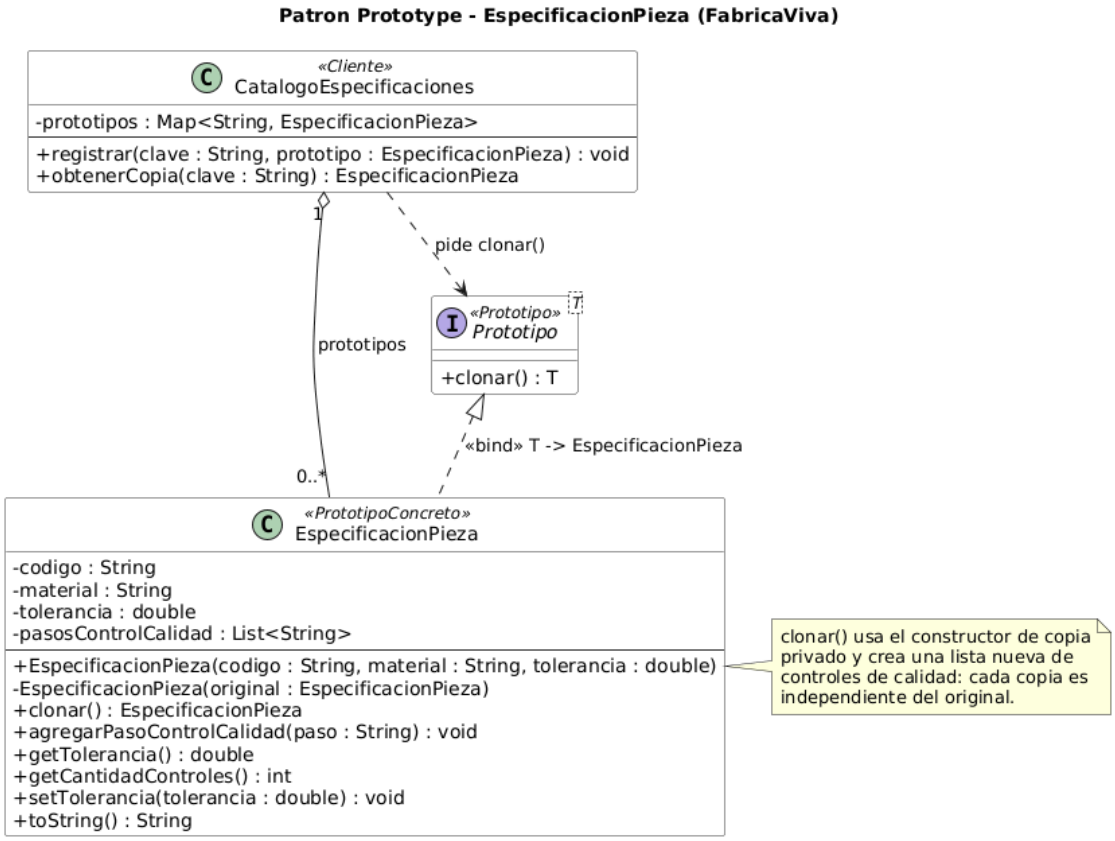
En FábricaViva, la interfaz Prototipo declara el método clonar(). EspecificacionPieza la implementa, y CatalogoEspecificaciones guarda las fichas maestras y entrega siempre una copia, nunca el original.

Diagrama UML

En la Figura 30, Prototipo<T> es una interfaz genérica y EspecificacionPieza la realiza sustituyendo T por su propio tipo (bind). La ficha tiene dos constructores: uno público, para crear fichas maestras, y uno privado (–) de copia, que solo usa clonar(). CatalogoEspecificaciones agrupa de cero a muchas especificaciones en su mapa prototipos y, cuando se le pide una, invoca clonar().

Figura 30

Diagrama de clases del patrón Prototype en FábricaViva



Nota. Elaborado en PlantUML a partir del archivo `docs/uml/05-prototype.puml`.

Implementación

La Figura 31 muestra la interfaz, que tiene una sola operación. En la Figura 32, clonar() llama al constructor de copia, que crea una lista nueva de controles de calidad en lugar de reutilizar la del original. Ese detalle es el que hace independiente a cada copia: si la lista se compartiera, agregar un control en una copia lo agregaría también a la ficha maestra.

Figura 31

Interfaz Prototipo

Prototipo.java

```java
package com.fabricaviva.prototipo;

/**
 * Patrón Prototype.
 * Declara la operación para clonarse (Gamma et al., 2003, p. 111).
 */
public interface Prototipo<T> {

    T clonar();
}
```

Nota. Captura del archivo `src/main/java/com/fabricaviva/prototipo/Prototipo.java` del repositorio del proyecto.

Figura 32

Prototipo concreto EspecificacionPieza

EspecificacionPieza.java

```java
package com.fabricaviva.prototipo;

import java.util.ArrayList;
import java.util.List;

/** Prototipo concreto: la ficha técnica de una pieza que la planta fabrica seguido. */
public class EspecificacionPieza implements Prototipo<EspecificacionPieza> {

    private final String codigo;
    private final String material;
    private double tolerancia;
    private final List<String> pasosControlCalidad;

    public EspecificacionPieza(String codigo, String material, double tolerancia) {
        this.codigo = codigo;
        this.material = material;
        this.tolerancia = tolerancia;
        this.pasosControlCalidad = new ArrayList<>();
    }

    // Constructor de copia: la lista se copia completa para que cada clon sea independiente.
    private EspecificacionPieza(EspecificacionPieza original) {
        this.codigo = original.codigo;
        this.material = original.material;
        this.tolerancia = original.tolerancia;
        this.pasosControlCalidad = new ArrayList<>(original.pasosControlCalidad);
    }

    @Override
    public EspecificacionPieza clonar() {
        return new EspecificacionPieza(this);
    }

    public void agregarPasoControlCalidad(String paso) {
        pasosControlCalidad.add(paso);
    }

    public double getTolerancia() {
        return tolerancia;
    }

    public int getCantidadControles() {
        return pasosControlCalidad.size();
    }

    public void setTolerancia(double tolerancia) {
        this.tolerancia = tolerancia;
    }

    @Override
    public String toString() {
        return codigo + " (" + material + ", tolerancia " + tolerancia + " mm) - controles: "
                + pasosControlCalidad;
    }
}
```

Nota. Captura del archivo `src/main/java/com/fabricaviva/prototipo/EspecificacionPieza.java` del repositorio del proyecto.

En la Figura 33, obtenerCopia() busca la ficha solicitada y devuelve su clon.

Figura 33

Clase CatalogoEspecificaciones

CatalogoEspecificaciones.java

```java
package com.fabricaviva.prototipo;

import java.util.HashMap;
import java.util.Map;

/** Guarda las especificaciones originales y entrega siempre una copia. */
public class CatalogoEspecificaciones {

    private final Map<String, EspecificacionPieza> prototipos = new HashMap<>();

    public void registrar(String clave, EspecificacionPieza prototipo) {
        prototipos.put(clave, prototipo);
    }

    public EspecificacionPieza obtenerCopia(String clave) {
        EspecificacionPieza prototipo = prototipos.get(clave);
        if (prototipo == null) {
            throw new IllegalArgumentException("No existe la especificacion: " + clave);
        }
        return prototipo.clonar();
    }
}
```

Nota. Captura del archivo `src/main/java/com/fabricaviva/prototipo/CatalogoEspecificaciones.java` del repositorio del proyecto.

Prueba de funcionamiento

PruebaPrototype (Figura 34) registra la ficha maestra del eje con dos controles de calidad y pide dos copias al catálogo. Después ajusta la copia 2 como lo pediría un cliente exigente: tolerancia de 0.02 mm y un ensayo de dureza adicional.

Figura 34

Clase PruebaPrototype

PruebaPrototype.java

```java
package com.fabricaviva;

import com.fabricaviva.prototipo.CatalogoEspecificaciones;
import com.fabricaviva.prototipo.EspecificacionPieza;

public class PruebaPrototype {

    public static void main(String[] args) {
        EspecificacionPieza ejeBase = new EspecificacionPieza("EJE-INOX-20", "Acero inoxidable", 0.05);
        ejeBase.agregarPasoControlCalidad("Verificar diametro");
        ejeBase.agregarPasoControlCalidad("Verificar acabado superficial");

        CatalogoEspecificaciones catalogo = new CatalogoEspecificaciones();
        catalogo.registrar("EJE-INOX-20", ejeBase);

        EspecificacionPieza copia1 = catalogo.obtenerCopia("EJE-INOX-20");
        EspecificacionPieza copia2 = catalogo.obtenerCopia("EJE-INOX-20");

        // Un cliente pide el eje con tolerancia mas estricta y una prueba adicional.
        copia2.setTolerancia(0.02);
        copia2.agregarPasoControlCalidad("Ensayo de dureza");

        System.out.println("== Prueba 1: el catalogo entrega copias listas para usar ==");
        System.out.println("Prototipo base: " + ejeBase);
        System.out.println("Copia 1       : " + copia1);
        System.out.println("Copia 2       : " + copia2);

        System.out.println("\n== Prueba 2: son objetos distintos en memoria ==");
        System.out.println("copia1 == copia2 ? " + (copia1 == copia2));
        System.out.println("copia1 == ejeBase ? " + (copia1 == ejeBase));

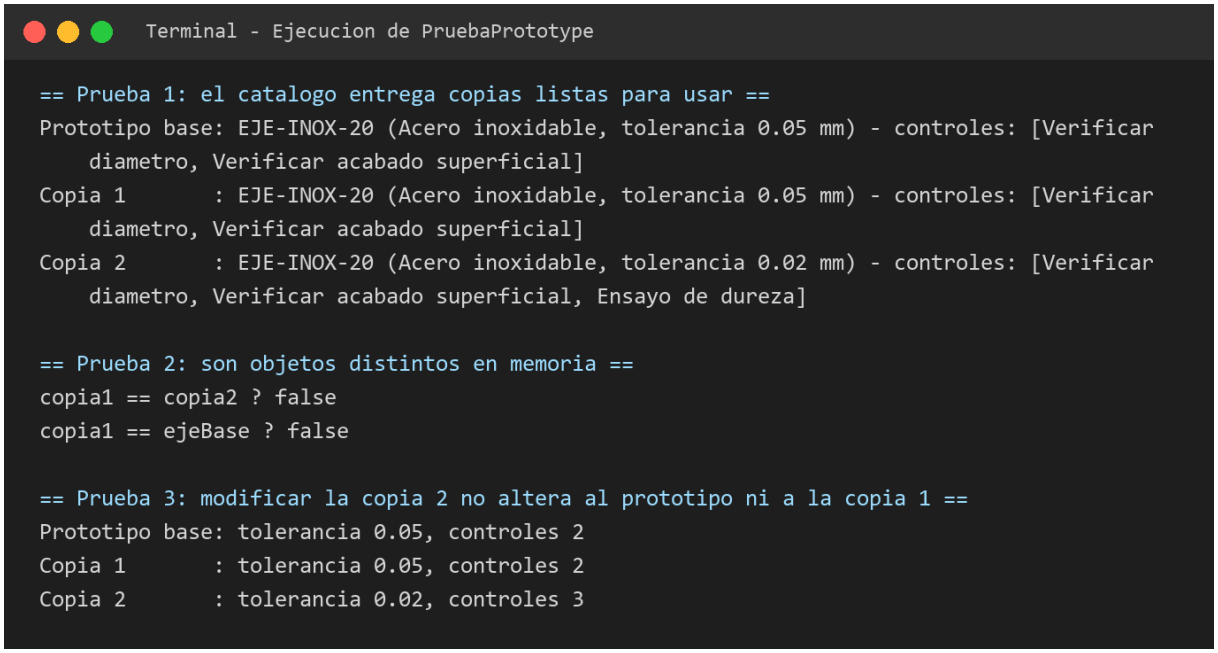
        System.out.println("\n== Prueba 3: modificar la copia 2 no altera al prototipo ni a la copia 1 ==");
        System.out.println("Prototipo base: tolerancia " + ejeBase.getTolerancia()
                + ", controles " + ejeBase.getCantidadControles());
        System.out.println("Copia 1       : tolerancia " + copia1.getTolerancia()
                + ", controles " + copia1.getCantidadControles());
        System.out.println("Copia 2       : tolerancia " + copia2.getTolerancia()
                + ", controles " + copia2.getCantidadControles());
    }
}
```

Nota. Captura del archivo `src/main/java/com/fabricaviva/PruebaPrototype.java` del repositorio del proyecto.

La Figura 35 muestra el resultado en tres partes. Las copias salen completas, con material, tolerancia y controles. Las comparaciones con == dan false, así que se trata de objetos distintos. Y lo más importante: la ficha maestra y la copia 1 conservan la tolerancia de 0.05 mm y sus 2 controles, mientras que solo la copia 2 tiene 0.02 mm y 3 controles. Esto demuestra que la copia es completa, incluida la lista de controles, y no un simple enlace al original.

Figura 35

Resultado de la ejecución de PruebaPrototype



Nota. Salida obtenida al ejecutar `PruebaPrototype`.

Patrones estructurales

Los patrones estructurales describen cómo componer clases y objetos para formar estructuras más grandes, manteniendo la flexibilidad y la eficiencia (Gamma et al., 2003, p. 155). En FábricaViva se implementan dos de ellos: Bridge y Adapter.

Patrón Bridge (AlertaProduccion)

Descripción del patrón

El patrón Bridge desacopla una abstracción de su implementación para que ambas puedan variar de forma independiente (Gamma et al., 2003, p. 141). En lugar de crear una jerarquía monolítica que combine el tipo de operación y el mecanismo de entrega, el patrón divide el diseño en dos jerarquías conectadas por una referencia: la abstracción conoce al implementador, pero no depende de ninguna clase concreta de éste.

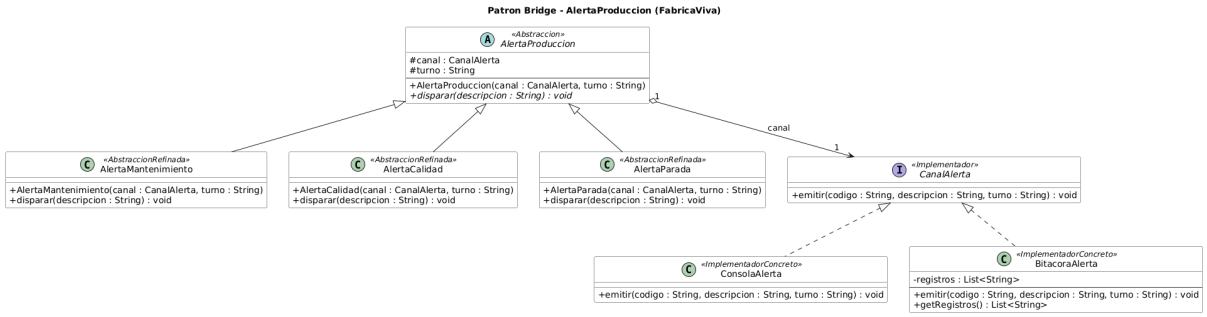
En FábricaViva, el MES necesita notificar a los operarios de tres tipos de eventos: averías de mantenimiento, desviaciones de calidad y paradas de línea no planificadas. Cada tipo puede emitirse por distintos canales: la consola del supervisor (ConsolaAlerta) o la bitácora persistente del turno (BitacoraAlerta). Sin Bridge, agregar un cuarto tipo de alerta obligaría a crear una clase por cada canal posible. Con Bridge, el tipo de alerta y el canal evolucionan de forma independiente, y cualquier combinación queda disponible sin código adicional.

Diagrama UML

La Figura 36 muestra la estructura del patrón Bridge. AlertaProduccion (abstracción) mantiene una referencia a CanalAlerta (implementador). AlertaMantenimiento, AlertaCalidad y AlertaParada refinan la abstracción; ConsolaAlerta y BitacoraAlerta proporcionan las implementaciones concretas.

Figura 36

Diagrama de clases del patrón Bridge en FábricaViva



Nota. Elaborado en PlantUML a partir del archivo docs/uml/06-bridge.puml del proyecto.

Implementación

La Figura 37 define la interfaz CanalAlerta con un único método emitir(), que recibe el código de la alerta, la descripción y el turno activo.

Figura 37

Interfaz CanalAlerta (implementador del Bridge)

CanalAlerta.java

```java
package com.fabricaviva.alertas;

/**
 * Patrón Bridge – rol Implementador.
 * Define cómo se entrega físicamente una alerta, sin importar su tipo.
 */
public interface CanalAlerta {

    void emitir(String codigo, String descripcion, String turno);
}
```

Nota. Captura del archivo src/main/java/com/fabricaviva/alertas/CanalAlerta.java del proyecto.

La Figura 38 muestra ConsolaAlerta, el implementador concreto que imprime la alerta por la salida estándar.

Figura 38

Implementador concreto ConsolaAlerta

ConsolaAlerta.java

```java
package com.fabricaviva.alertas;

/**
 * Patrón Bridge – Implementador Concreto.
 * Muestra la alerta por la salida estándar (consola del operario).
 */
public class ConsolaAlerta implements CanalAlerta {

    @Override
    public void emitir(String codigo, String descripcion, String turno) {
        System.out.println("[ALERTA] " + codigo + " | " + descripcion + " | Turno: " + turno);
    }
}
```

Nota. Captura del archivo src/main/java/com/fabricaviva/alertas/ConsolaAlerta.java del proyecto.

La Figura 39 presenta BitacoraAlerta, el implementador que acumula los registros en memoria y los expone de forma inmutable para auditoría del turno.

Figura 39

Implementador concreto BitacoraAlerta

BitacoraAlerta.java

```java
package com.fabricaviva.alertas;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * Patrón Bridge – Implementador Concreto.
 * Persiste la alerta en una bitácora en memoria (simula registro persistente).
 */
public class BitacoraAlerta implements CanalAlerta {

    private final List<String> registros = new ArrayList<>();

    @Override
    public void emitir(String codigo, String descripcion, String turno) {
        String entrada = codigo + " | " + descripcion + " | Turno: " + turno;
        registros.add(entrada);
        System.out.println("[BITACORA] Registrado: " + entrada);
    }

    public List<String> getRegistros() {
        return Collections.unmodifiableList(registros);
    }
}
```

Nota. Captura del archivo src/main/java/com/fabricaviva/alertas/BitacoraAlerta.java del proyecto.

La Figura 40 define AlertaProduccion, la abstracción que agrupa el turno y el canal, y declara el método abstracto disparar(). Las subclases solo necesitan proporcionar el código y el prefijo de la alerta.

Figura 40

Abstracción AlertaProduccion

AlertaProduccion.java

```java
package com.fabricaviva.alertas;

/**
 * Patrón Bridge – Abstracción.
 * Define qué se reporta (tipo de alerta) y delega cómo se entrega al canal.
 * Abstracción e implementación pueden variar de forma independiente.
 */
public abstract class AlertaProduccion {

    protected final CanalAlerta canal;
    protected final String turno;

    public AlertaProduccion(CanalAlerta canal, String turno) {
        this.canal = canal;
        this.turno = turno;
    }

    public abstract void disparar(String descripcion);
}
```

Nota. Captura del archivo src/main/java/com/fabricaviva/alertas/AlertaProduccion.java del proyecto.

La Figura 41 ilustra AlertaMantenimiento, abstracción refinada representativa del patrón. AlertaCalidad y AlertaParada siguen la misma estructura, diferenciándose únicamente en el código y el prefijo descriptivo.

Figura 41

Abstracción refinada AlertaMantenimiento (representativa)

AlertaMantenimiento.java

```java
package com.fabricaviva.alertas;

/**
 * Patrón Bridge – Abstracción Refinada.
 * Alerta de mantenimiento preventivo o correctivo sobre un equipo de la planta.
 */
public class AlertaMantenimiento extends AlertaProduccion {

    public AlertaMantenimiento(CanalAlerta canal, String turno) {
        super(canal, turno);
    }

    @Override
    public void disparar(String descripcion) {
        canal.emitir("MAN-001", "[MANTENIMIENTO] " + descripcion, turno);
    }
}
```

Nota. Captura del archivo src/main/java/com/fabricaviva/alertas/AlertaMantenimiento.java del proyecto.

Prueba de funcionamiento

PruebaBridge (Figura 42) crea dos canales y combina con tres tipos de alerta. La prueba 4 demuestra el beneficio central del patrón: el mismo tipo de alerta (AlertaMantenimiento) se redirige al canal BitacoraAlerta sin modificar ninguna de las dos jerarquías.

Figura 42

Clase PruebaBridge

PruebaBridge.java

```java
package com.fabricaviva;

import com.fabricaviva.alertas.AlertaCalidad;
import com.fabricaviva.alertas.AlertaMantenimiento;
import com.fabricaviva.alertas.AlertaParada;
import com.fabricaviva.alertas.AlertaProduccion;
import com.fabricaviva.alertas.BitacoraAlerta;
import com.fabricaviva.alertas.ConsolaAlerta;

public class PruebaBridge {

    public static void main(String[] args) {

        ConsolaAlerta consola = new ConsolaAlerta();
        BitacoraAlerta bitacora = new BitacoraAlerta();

        System.out.println("== Prueba 1: alerta de mantenimiento por consola ==");
        AlertaProduccion mant = new AlertaMantenimiento(consola, "MANANA");
        mant.disparar("Motor CNC-03 requiere lubricacion");

        System.out.println("\n== Prueba 2: alerta de calidad por consola ==");
        AlertaProduccion cal = new AlertaCalidad(consola, "MANANA");
        cal.disparar("Tasa de rechazo supera 3% en linea L-02");

        System.out.println("\n== Prueba 3: parada de linea registrada en bitacora ==");
        AlertaProduccion parada = new AlertaParada(bitacora, "TARDE");
        parada.disparar("L-01 detenida por falla mecanica");

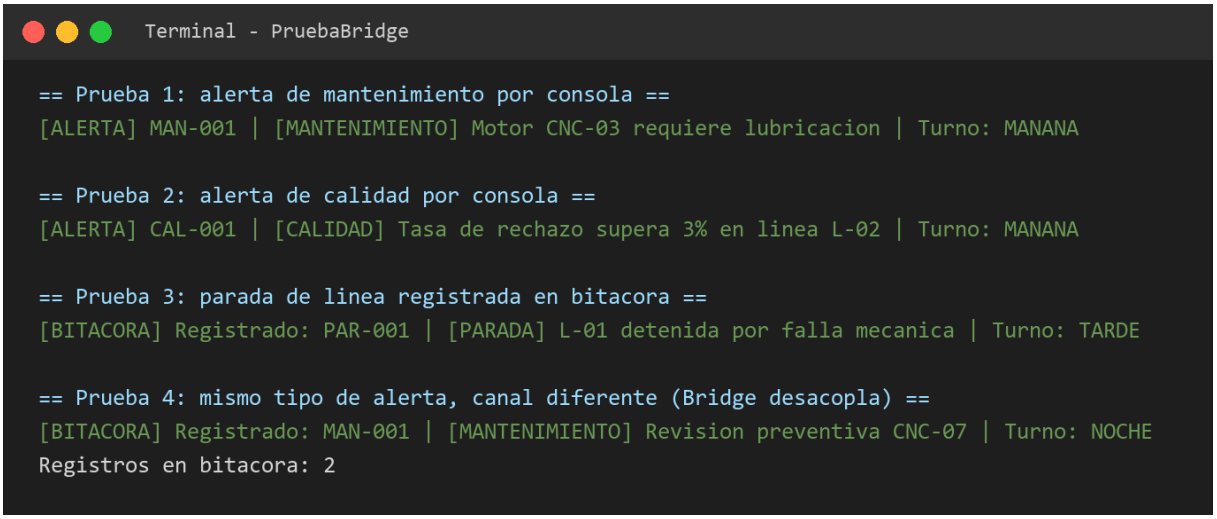
        System.out.println("\n== Prueba 4: mismo tipo de alerta, canal diferente (Bridge desacopla) ==");
        AlertaProduccion mantBitacora = new AlertaMantenimiento(bitacora, "NOCHE");
        mantBitacora.disparar("Revision preventiva CNC-07");
        System.out.println("Registros en bitacora: " + bitacora.getRegistros().size());
    }
}
```

Nota. Captura del archivo src/main/java/com/fabricaviva/PruebaBridge.java del proyecto.

La Figura 43 registra la salida de la prueba. Las cuatro alertas se emiten correctamente por el canal asignado y la bitácora acumula dos entradas al finalizar la ejecución.

Figura 43

Resultado de la ejecución de PruebaBridge



Nota. Salida obtenida al ejecutar PruebaBridge.

Patrón Adapter (AdaptadorErp)

Descripción del patrón

El patrón Adapter convierte la interfaz de una clase en otra que los clientes esperan, permitiendo la cooperación entre clases con interfaces incompatibles (Gamma et al., 2003, p. 131). Actúa como un puente de traducción: el cliente solo conoce la interfaz objetivo (Target) y delega en el adaptador, que internamente reformatea las llamadas al objeto adaptado (Adaptee).

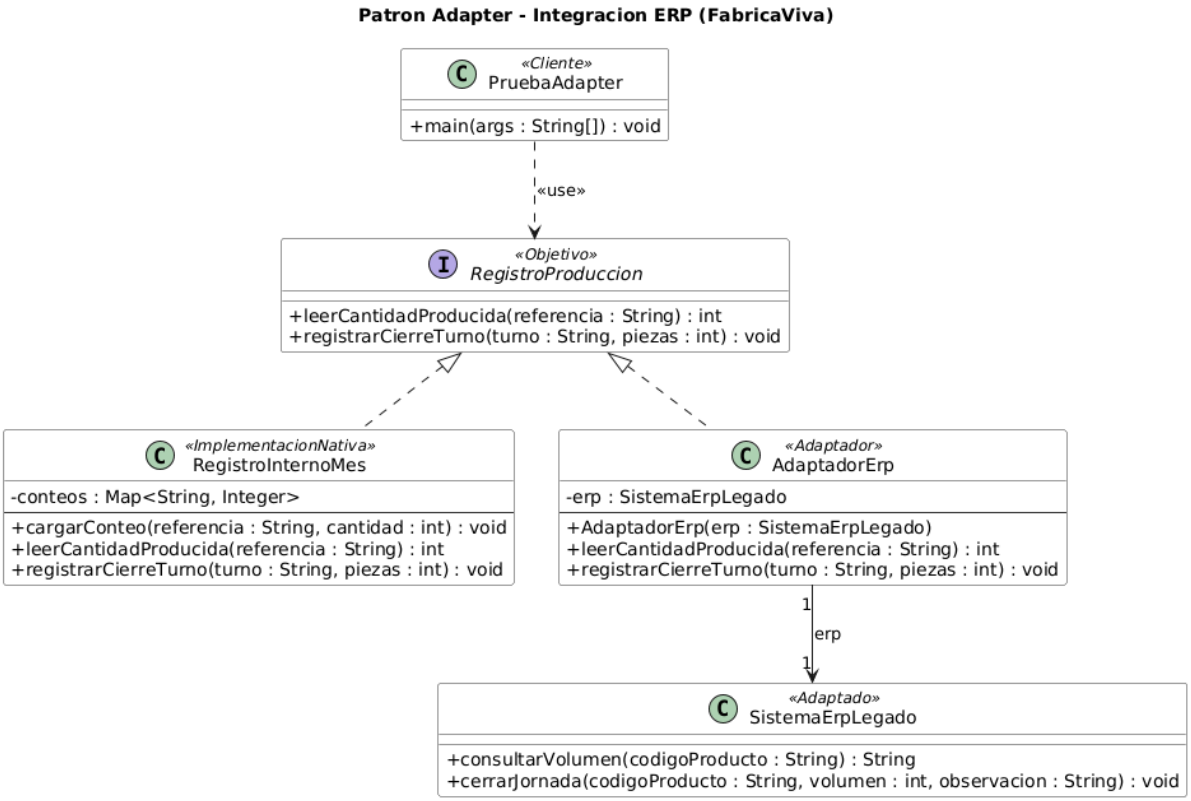
En FábricaViva, el MES necesita integrar un ERP heredado (SistemaErpLegado) que expone métodos con firmas incompatibles con el contrato del sistema. El ERP utiliza consultarVolumen(), que retorna una cadena con unidades ('312 piezas'), mientras que el MES espera leerCantidadProducida(), que devuelve un entero. AdaptadorErp implementa la interfaz RegistroProduccion y realiza la traducción necesaria sin modificar el MES ni el sistema heredado.

Diagrama UML

La Figura 44 presenta la estructura del patrón Adapter. RegistroProduccion define el contrato que el MES utiliza. RegistroInternoMes lo implementa de forma nativa; AdaptadorErp también lo implementa, pero delega cada llamada al SistemaErpLegado realizando la traducción necesaria.

Figura 44

Diagrama de clases del patrón Adapter en FábricaViva



Nota. Elaborado en PlantUML a partir del archivo docs/uml/07-adapter.puml del proyecto.

Implementación

La Figura 45 muestra la interfaz RegistroProduccion, el contrato objetivo que el MES emplea para leer datos de producción y registrar el cierre de turno, independientemente del sistema subyacente.

Figura 45

Interfaz RegistroProduccion (Target del Adapter)

RegistroProduccion.java

```java
package com.fabricaviva.integracion;

/**
 * Patrón Adapter – interfaz Objetivo (Target).
 * Contrato que el MES FábricaViva utiliza para leer y cerrar registros
 * de producción, independientemente del sistema subyacente.
 */
public interface RegistroProduccion {

    int leerCantidadProducida(String referencia);

    void registrarCierreTurno(String turno, int piezas);
}
```

Nota. Captura del archivo src/main/java/com/fabricaviva/integracion/RegistroProduccion.java del proyecto.

La Figura 46 reproduce SistemaErpLegado, el objeto adaptado (Adaptee) con su API incompatible. Al ser un sistema externo, no puede modificarse.

Figura 46

Clase SistemaErpLegado (Adaptee)

SistemaErpLegado.java

```java
package com.fabricaviva.integracion;

/**
 * Patrón Adapter – rol Adaptado (Adaptee).
 * ERP heredado con API incompatible con el contrato del MES.
 * No puede modificarse porque pertenece a un proveedor externo.
 */
public class SistemaErpLegado {

    public String consultarVolumen(String codigoProducto) {
        System.out.println("Consultando ERP... respuesta: \"312 piezas\"");
        return "312 piezas";
    }

    public void cerrarJornada(String codigoProducto, int volumen, String observacion) {
        System.out.println("ERP: jornada cerrada -> " + codigoProducto
                + " | vol=" + volumen + " | obs=" + observacion);
    }
}
```

Nota. Captura del archivo src/main/java/com/fabricaviva/integracion/SistemaErpLegado.java del proyecto.

La Figura 47 contiene AdaptadorErp, que implementa RegistroProduccion y traduce cada llamada: leerCantidadProducida() invoca consultarVolumen() y convierte la cadena al entero esperado; registrarCierreTurno() invoca cerrarJornada() con los parámetros correctos.

Figura 47

Clase AdaptadorErp (Adaptador)

AdaptadorErp.java

```java
package com.fabricaviva.integracion;

/**
 * Patrón Adapter – rol Adaptador.
 * Traduce las llamadas del MES (RegistroProduccion) al protocolo
 * del ERP legado (SistemaErpLegado) sin alterar ninguno de los dos.
 */
public class AdaptadorErp implements RegistroProduccion {

    private final SistemaErpLegado erp;

    public AdaptadorErp(SistemaErpLegado erp) {
        this.erp = erp;
    }

    @Override
    public int leerCantidadProducida(String referencia) {
        String respuesta = erp.consultarVolumen(referencia);
        return Integer.parseInt(respuesta.split(" ")[0]);
    }

    @Override
    public void registrarCierreTurno(String turno, int piezas) {
        erp.cerrarJornada(turno, piezas, "Cierre automatico MES-FabricaViva");
    }
}
```

Nota. Captura del archivo src/main/java/com/fabricaviva/integracion/AdaptadorErp.java del proyecto.

Prueba de funcionamiento

PruebaAdapter (Figura 48) instancia primero el registro nativo del MES y luego el adaptador del ERP. En la prueba 3 se itera sobre un arreglo de RegistroProduccion para demostrar que el cliente no distingue entre las dos implementaciones: ambas responden con 312 piezas.

Figura 48

Clase PruebaAdapter

PruebaAdapter.java

```java
package com.fabricaviva;

import com.fabricaviva.integracion.AdaptadorErp;
import com.fabricaviva.integracion.RegistroInternoMes;
import com.fabricaviva.integracion.RegistroProduccion;
import com.fabricaviva.integracion.SistemaErpLegado;

public class PruebaAdapter {

    public static void main(String[] args) {

        System.out.println("== Prueba 1: registro interno MES (implementacion nativa) ==");
        RegistroInternoMes mesNativo = new RegistroInternoMes();
        mesNativo.cargarConteo("REF-A220", 312);
        int cantidadMes = mesNativo.leerCantidadProducida("REF-A220");
        System.out.println("Cantidad producida (MES): " + cantidadMes + " piezas");
        mesNativo.registrarCierreTurno("MANANA", cantidadMes);

        System.out.println("\n== Prueba 2: adaptador ERP conectado al mismo contrato ==");
        SistemaErpLegado erpLegado = new SistemaErpLegado();
        RegistroProduccion adaptador = new AdaptadorErp(erpLegado);
        int cantidadErp = adaptador.leerCantidadProducida("REF-A220");
        System.out.println("Cantidad producida (ERP): " + cantidadErp + " piezas");
        adaptador.registrarCierreTurno("MANANA", cantidadErp);

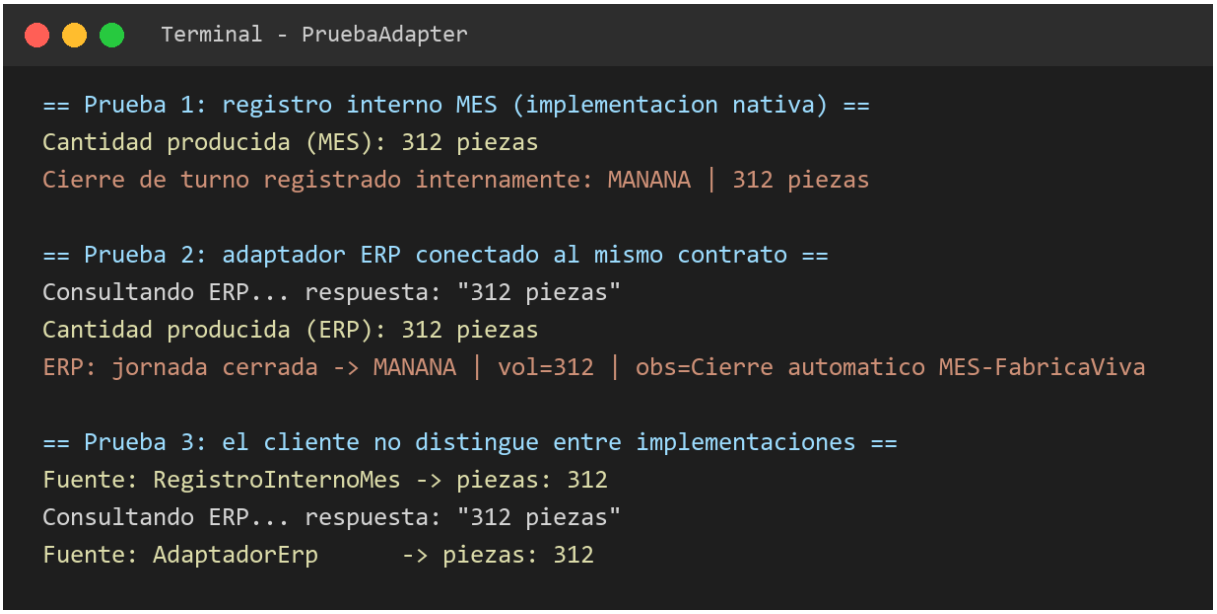
        System.out.println("\n== Prueba 3: el cliente no distingue entre implementaciones ==");
        RegistroProduccion[] fuentes = { mesNativo, adaptador };
        String[] nombres = { "RegistroInternoMes", "AdaptadorErp     " };
        for (int i = 0; i < fuentes.length; i++) {
            int piezas = fuentes[i].leerCantidadProducida("REF-A220");
            System.out.println("Fuente: " + nombres[i] + " -> piezas: " + piezas);
        }
    }
}
```

Nota. Captura del archivo src/main/java/com/fabricaviva/PruebaAdapter.java del proyecto.

La Figura 49 muestra la ejecución completa. El adaptador traduce de forma transparente la respuesta del ERP y la prueba 3 confirma que ambas fuentes devuelven el mismo valor entero al cliente.

Figura 49

Resultado de la ejecución de PruebaAdapter



Nota. Salida obtenida al ejecutar PruebaAdapter.

Patrón Composite (Jerarquía de Unidades de Producción)

Descripción del patrón

El patrón Composite compone objetos en estructuras de árbol para representar jerarquías de parte-todo, de modo que los clientes traten objetos individuales y composiciones de forma uniforme (Gamma et al., 2003, p. 163). Tanto la hoja como el contenedor implementan la misma interfaz, por lo que el cliente invoca los mismos métodos sin necesidad de instanceof.

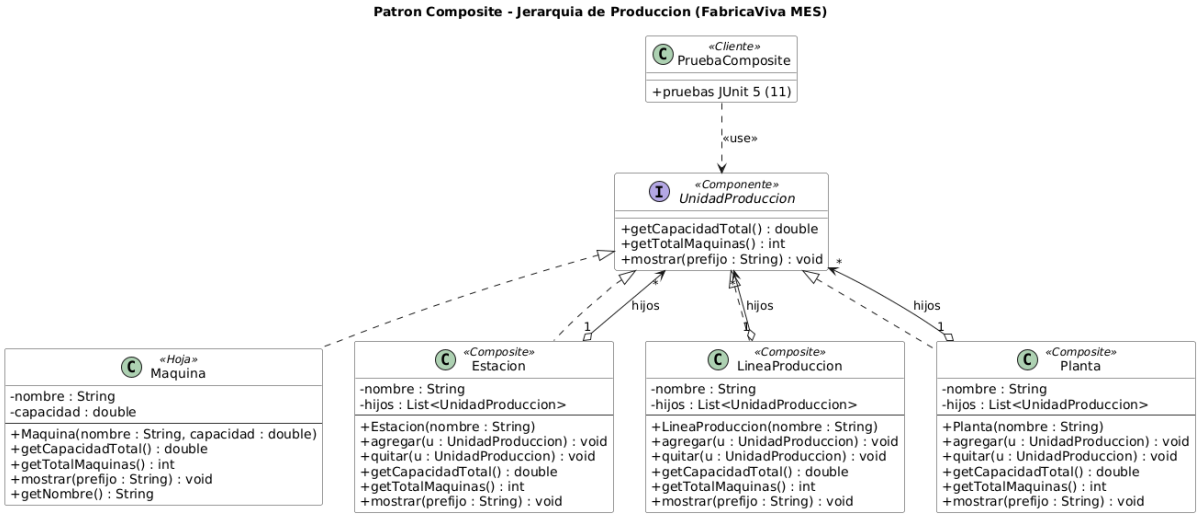
En FábricaViva, el módulo de gestión de planta organiza los recursos de producción en una jerarquía de cuatro niveles: Máquina (hoja) → Estación → Línea de producción → Planta. La interfaz UnidadProduccion define el contrato uniforme con tres operaciones: getCapacidadTotal(), getTotalMaquinas() y mostrar(). Cualquier módulo del sistema —reportes de capacidad, planificación de turnos, mantenimiento— puede consultar la capacidad o el número de máquinas de cualquier nodo sin conocer su tipo concreto.

Diagrama UML

La Figura 50 muestra la estructura del patrón Composite en el dominio del MES. Maquina implementa UnidadProduccion sin hijos; Estacion, LineaProduccion y Planta son los composites que mantienen una lista de UnidadProduccion y delegan recursivamente las operaciones.

Figura 50

Diagrama de clases del patrón Composite (jerarquía de unidades de producción)



Nota. Elaborado en PlantUML a partir del archivo docs/uml/08-composite.puml del proyecto.

Implementación

La Figura 51 define la interfaz UnidadProduccion, contrato uniforme que deben cumplir tanto las hojas como los composites del árbol de producción.

Figura 51

Interfaz UnidadProduccion (Componente del Composite)

UnidadProduccion.java

```java
package com.fabricaviva.produccion;

/**
 * Componente del Composite: contrato uniforme para hojas y contenedores
 * de la jerarquía de unidades de producción (Gamma et al., 2003, p. 163).
 */
public interface UnidadProduccion {
    /** Capacidad nominal total en piezas/hora. */
    double getCapacidadTotal();

    /** Número total de máquinas individuales en el subárbol. */
    int getTotalMaquinas();

    /** Visualiza la estructura con sangría. */
    void mostrar(String prefijo);
}
```

Nota. Captura del archivo src/main/java/com/fabricaviva/produccion/UnidadProduccion.java.

La Figura 52 presenta Maquina, la hoja del árbol. Almacena la capacidad nominal individual y retorna 1 en getTotalMaquinas().

Figura 52

Clase Maquina (hoja del Composite)

Maquina.java

```java
package com.fabricaviva.produccion;

/** Hoja del Composite: máquina individual con capacidad fija. */
public class Maquina implements UnidadProduccion {

    private final String nombre;
    private final double capacidad;

    public Maquina(String nombre, double capacidad) {
        this.nombre    = nombre;
        this.capacidad = capacidad;
    }

    @Override
    public double getCapacidadTotal() {
        return capacidad;
    }

    @Override
    public int getTotalMaquinas() {
        return 1;
    }

    @Override
    public void mostrar(String prefijo) {
        System.out.printf("%s[Máquina] %s — %.0f pz/h%n", prefijo, nombre, capacidad);
    }

    public String getNombre() { return nombre; }
}
```

Nota. Captura del archivo src/main/java/com/fabricaviva/produccion/Maquina.java.

La Figura 53 muestra Estacion, composite concreto representativo. Estacion, LineaProduccion y Planta comparten la misma estructura: lista de hijos y delegación recursiva.

getCapacidadTotal() acumula la suma de sus hijos; getTotalMaquinas() suma el recuento recursivo.

Figura 53

Clase Estacion (Composite concreto representativo)

Estacion.java

```java
package com.fabricaviva.produccion;

import java.util.ArrayList;
import java.util.List;

/** Composite concreto: estación de trabajo que agrupa máquinas. */
public class Estacion implements UnidadProduccion {

    private final String nombre;
    private final List<UnidadProduccion> hijos = new ArrayList<>();

    public Estacion(String nombre) { this.nombre = nombre; }

    public void agregar(UnidadProduccion u) { hijos.add(u); }
    public void quitar(UnidadProduccion u)  { hijos.remove(u); }

    @Override
    public double getCapacidadTotal() {
        return hijos.stream().mapToDouble(UnidadProduccion::getCapacidadTotal).sum();
    }

    @Override
    public int getTotalMaquinas() {
        return hijos.stream().mapToInt(UnidadProduccion::getTotalMaquinas).sum();
    }

    @Override
    public void mostrar(String prefijo) {
        System.out.printf("%s[Estación] %s%n", prefijo, nombre);
        hijos.forEach(h -> h.mostrar(prefijo + "  "));
    }
}
```

Nota. Captura del archivo src/main/java/com/fabricaviva/produccion/Estacion.java.

Prueba de funcionamiento

PruebaComposite (Figuras 54 y 55) contiene 11 pruebas JUnit 5 que validan la capacidad y el recuento de máquinas a los cuatro niveles del árbol, la uniformidad de trato entre hoja y composite, y el comportamiento dinámico al agregar o quitar máquinas en tiempo de ejecución.

Figura 54

Clase PruebaComposite — primera parte (setUp y pruebas Maquina y Estacion)

PruebaComposite.java

```java
package com.fabricaviva.pruebas;

import com.fabricaviva.produccion.*;
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

/**
 * Verifica que el patrón Composite permita tratar de forma uniforme
 * las hojas (Maquina) y los contenedores (Estacion, LineaProduccion, Planta).
 */
class PruebaComposite {

    private Maquina tornoCNC;
    private Maquina fresadora;
    private Maquina soldadora;
    private Estacion estacion1;
    private Estacion estacion2;
    private LineaProduccion linea;
    private Planta planta;

    @BeforeEach
    void setUp() {
        tornoCNC  = new Maquina("Torno CNC",   120.0);
        fresadora = new Maquina("Fresadora",    80.0);
        soldadora = new Maquina("Soldadora",   100.0);

        estacion1 = new Estacion("Mecanizado");
        estacion1.agregar(tornoCNC);
        estacion1.agregar(fresadora);

        estacion2 = new Estacion("Ensamble");
        estacion2.agregar(soldadora);

        linea = new LineaProduccion("Línea A");
        linea.agregar(estacion1);
        linea.agregar(estacion2);

        planta = new Planta("Planta FábricaViva");
        planta.agregar(linea);
    }

    // --- Maquina (hoja) ---

    @Test
    void maquinaIndividualReportaCapacidadPropia() {
        assertEquals(120.0, tornoCNC.getCapacidadTotal(), 0.001);
    }

    @Test
    void maquinaIndividualReportaUnaMaquina() {
        assertEquals(1, tornoCNC.getTotalMaquinas());
    }
```

Nota. Captura del archivo src/main/java/com/fabricaviva/pruebas/PruebaComposite.java.

Figura 55

Clase PruebaComposite — segunda parte (pruebas Linea, Planta y modificación dinámica)

PruebaComposite.java

```java
    @Test
    void estacionSumaCapacidadDeSusHijos() {
        // 120 + 80 = 200
        assertEquals(200.0, estacion1.getCapacidadTotal(), 0.001);
    }

    @Test
    void estacionCuentaTotalMaquinas() {
        assertEquals(2, estacion1.getTotalMaquinas());
    }

    // --- LineaProduccion (composite nivel 2) ---

    @Test
    void lineaSumaCapacidadDeSusEstaciones() {
        // (120+80) + 100 = 300
        assertEquals(300.0, linea.getCapacidadTotal(), 0.001);
    }

    @Test
    void lineaCuentaTotalMaquinasRecursivo() {
        assertEquals(3, linea.getTotalMaquinas());
    }

    // --- Planta (composite raíz) ---

    @Test
    void plantaSumaCapacidadDeTodaLaJerarquia() {
        assertEquals(300.0, planta.getCapacidadTotal(), 0.001);
    }

    @Test
    void plantaCuentaTodosLosMaquinasDeLaJerarquia() {
        assertEquals(3, planta.getTotalMaquinas());
    }

    // --- Uniformidad de trato ---

    @Test
    void clienteTrataMaquinaYPlantaMedianteMismaInterfaz() {
        UnidadProduccion hoja      = tornoCNC;
        UnidadProduccion compuesto = planta;
```

Nota. Captura del archivo src/main/java/com/fabricaviva/pruebas/PruebaComposite.java.

Patrón Decorator (Cálculo del Costo de Órdenes de Producción)

Descripción del patrón

El patrón Decorator agrega responsabilidades adicionales a un objeto de forma dinámica, proporcionando una alternativa flexible a la subclasificación para extender la funcionalidad (Gamma et al., 2003, p. 175). El decorador envuelve al componente, implementa la misma interfaz y delega en él, aplicando su propio ajuste antes o después de la delegación.

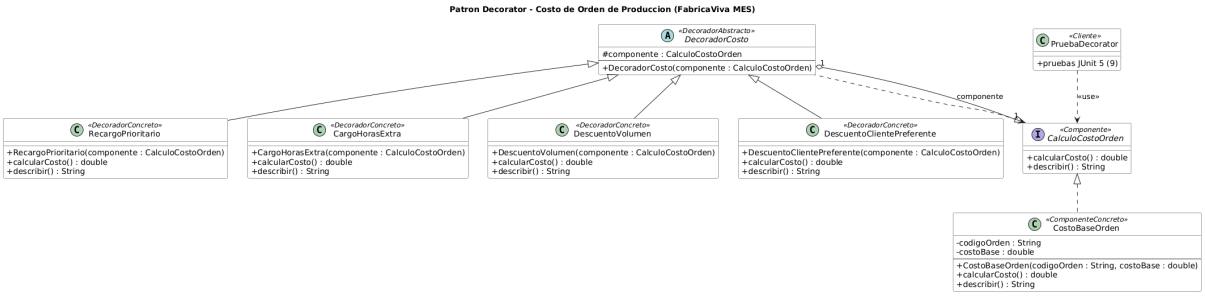
En FábricaViva, el cálculo del costo de una orden de producción parte de un costo base definido en el contrato con el cliente. Sobre ese valor pueden aplicarse en cualquier combinación: recargo por prioridad (+25 %), cargo por turno en horas extra (+30 %), descuento por alto volumen (−15 %) y descuento por cliente preferente (−10 %). Con herencia, cuatro ajustes requerirían $2^4 = 16$ subclases; con Decorator, son cuatro clases independientes y cualquier combinación se construye en tiempo de ejecución sin modificar el código existente.

Diagrama UML

La Figura 56 muestra la estructura del patrón Decorator en el MES. CostoBaseOrden implementa CalculoCostoOrden directamente; DecoradorCosto también lo implementa y mantiene una referencia al componente envuelto. Los cuatro decoradores concretos extienden DecoradorCosto.

Figura 56

Diagrama de clases del patrón Decorator (costo de orden de producción)



Nota. Elaborado en PlantUML a partir del archivo docs/uml/09-decorator.puml del proyecto.

Implementación

La Figura 57 define CalculoCostoOrden, interfaz del patrón. Declara calcularCosto() para el monto final y describir() para auditar los ajustes aplicados.

Figura 57

Interfaz CalculoCostoOrden y clase CostoBaseOrden (Componente concreto)

CalculoCostoOrden.java

```java
package com.fabricaviva.costos;

/**
 * Componente del Decorator: contrato para calcular el costo de una
 * orden de producción con ajustes combinables (Gamma et al., 2003, p. 175).
 */
public interface CalculoCostoOrden {
    /** Costo total de la orden tras aplicar todos los ajustes. */
    double calcularCosto();

    /** Descripción auditada de los ajustes aplicados. */
    String describir();
}
```

Nota. Captura del archivo src/main/java/com/fabricaviva/costos/CalculoCostoOrden.java.

La Figura 58 muestra DecoradorCosto, decorador abstracto, y RecargoPrioritario, decorador concreto representativo que aplica un multiplicador de 1.25. Los demás decoradores siguen la misma estructura.

Figura 58

DecoradorCosto (abstracto) y RecargoPrioritario (decorador concreto representativo)

RecargoPrioritario.java

```java
package com.fabricaviva.costos;

/** Recargo del +25 % por orden prioritaria de producción. */
public class RecargoPrioritario extends DecoradorCosto {

    public RecargoPrioritario(CalculoCostoOrden componente) {
        super(componente);
    }

    @Override
    public double calcularCosto() {
        return componente.calcularCosto() * 1.25;
    }

    @Override
    public String describir() {
        return componente.describir() + " + recargo prioritario (25%)";
    }
}
```

Nota. Captura del archivo src/main/java/com/fabricaviva/costos/RecargoPrioritario.java.

Prueba de funcionamiento

PruebaDecorator (Figura 59) contiene 9 pruebas JUnit 5 que validan: el costo base sin decoradores, cada decorador de forma individual, dos decoradores en distinto orden (conmutatividad de factores multiplicativos), tres decoradores encadenados y la acumulación de la descripción en cada capa.

Figura 59

Clase PruebaDecorator (9 pruebas JUnit 5)

PruebaDecorator.java

```java
package com.fabricaviva.pruebas;

import com.fabricaviva.costos.*;
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

/**
 * Verifica que el patrón Decorator permita combinar ajustes de costo
 * de órdenes de producción sin modificar las clases existentes.
 */
class PruebaDecorator {

    private static final double COSTO_BASE = 1_000_000.0;
    private static final double DELTA      = 0.01;

    private CalculoCostoOrden base;

    @BeforeEach
    void setUp() {
        base = new CostoBaseOrden("OP-2024-001", COSTO_BASE);
    }

    @Test
    void costoBaseSinDecoradores() {
        assertEquals(COSTO_BASE, base.calcularCosto(), DELTA);
    }

    @Test
    void recargoPrioritarioAumentaCosto25Porciento() {
        CalculoCostoOrden orden = new RecargoPrioritario(base);
        assertEquals(COSTO_BASE * 1.25, orden.calcularCosto(), DELTA);
    }

    @Test
    void cargoHorasExtraAumentaCosto30Porciento() {
        CalculoCostoOrden orden = new CargoHorasExtra(base);
        assertEquals(COSTO_BASE * 1.30, orden.calcularCosto(), DELTA);
    }

    @Test
    void descuentoVolumenReduceCosto15Porciento() {
        CalculoCostoOrden orden = new DescuentoVolumen(base);
        assertEquals(COSTO_BASE * 0.85, orden.calcularCosto(), DELTA);
    }

    @Test
    void descuentoClientePreferenteReduceCosto10Porciento() {
        CalculoCostoOrden orden = new DescuentoClientePreferente(base);
        assertEquals(COSTO_BASE * 0.90, orden.calcularCosto(), DELTA);
    }

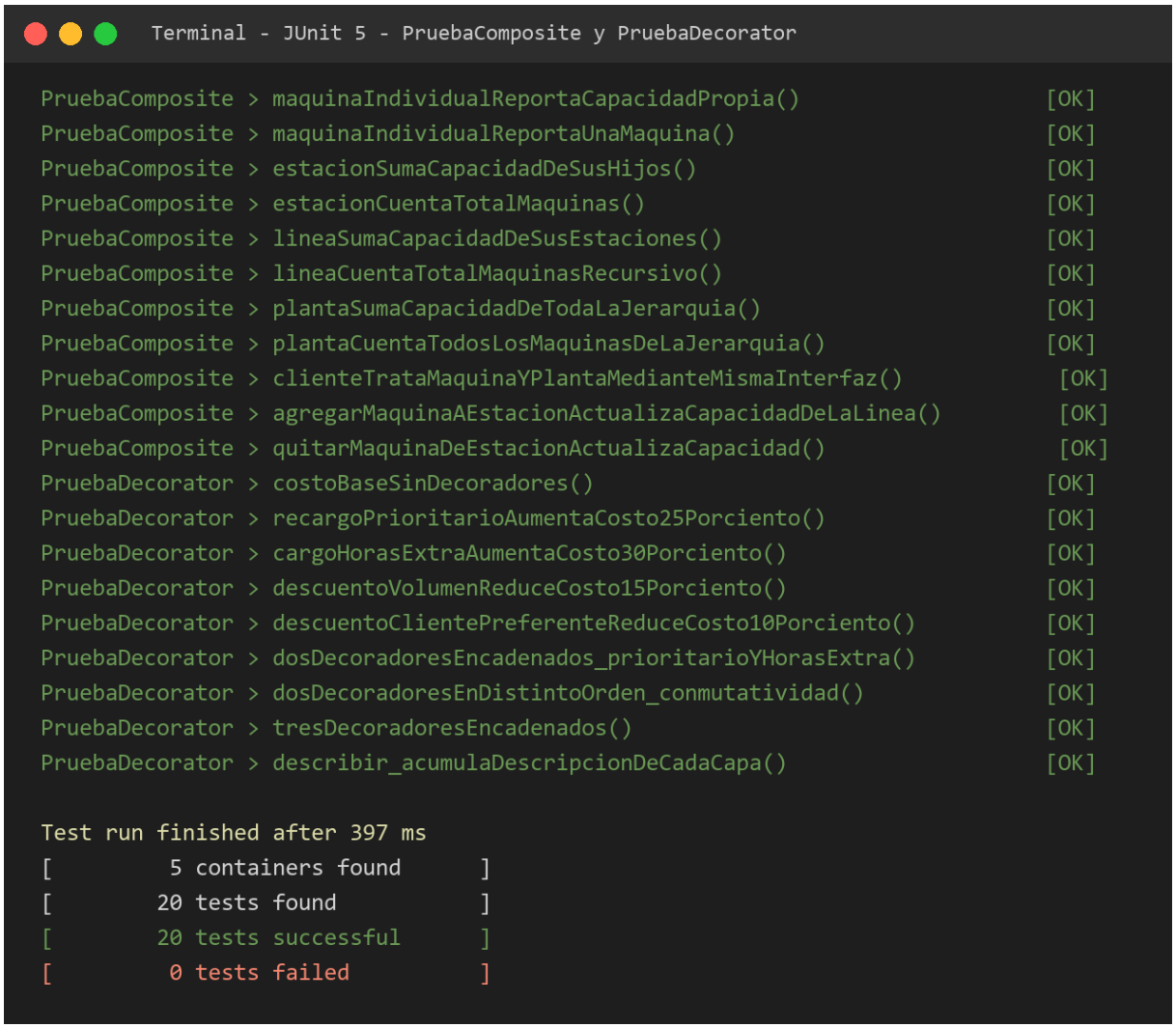
    @Test
    void dosDecoradoresEncadenados_prioritarioYHorasExtra() {
        // 1 000 000 * 1.25 * 1.30 = 1 625 000
        CalculoCostoOrden orden = new CargoHorasExtra(new RecargoPrioritario(base));
        assertEquals(COSTO_BASE * 1.25 * 1.30, orden.calcularCosto(), DELTA);
    }
```

Nota. Captura del archivo src/main/java/com/fabricaviva/pruebas/PruebaDecorator.java.

La Figura 60 muestra la ejecución de las 20 pruebas JUnit 5 (11 de Composite y 9 de Decorator). Todas pasan con éxito, sin errores ni omisiones.

Figura 60

Resultado de la ejecución de las 20 pruebas JUnit 5 (20/20 exitosas)



Nota. Salida obtenida ejecutando JUnit Platform Console Standalone 1.10.3.

Referencias

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (2003). *Patrones de diseño: Elementos de software orientado a objetos reutilizable*. Pearson Educación.

Medina Veloz, G., Luna Rosas, F. J., Muñoz Arteaga, J., & Martínez Romo, J. C. (2008). Integrando mediante patrones de software una estrategia fuzzy-logic, en un servicio de balanceo dinámico de carga bajo CORBA. *Conciencia Tecnológica*, (35), 12-20. http://www.redalyc.org/articulo.oa?id=94403502

MESA International. (1997). *MES explained: A high level vision* (White Paper N.º 6). Manufacturing Execution Systems Association. https://www.pathlms.com/mesa/courses/24315

Nakajima, S. (1988). *Introduction to TPM: Total productive maintenance*. Productivity Press.